

SBD - Plataforma analítica Cloud en AWS con integración batch, analítica y minería de datos

Alberto Piedra Venegas

March 2026

Índice

1. Introducción	3
1.1. Contextualización	3
1.2. Justificación	3
1.3. Objetivos	3
1.4. Alcance del proyecto	4
1.5. Planificación	4
1.5.1. Primera fase: Analisis del diseño - 1 día	4
1.5.2. Segunda fase: Implementación de la estructura base - 1 día	5
1.5.3. Tercera fase: Implementación de la ETL - 1 día	5
1.5.4. Cuarta fase: Procesamiento distribuido con Amazon EMR - 1 día	5
1.5.5. Quinta fase: Visualización de resultados - 1 día	5
1.5.6. Sexta fase: Documentación - 3 día	5
2. Marco teórico (Cloud Computing y Big Data en AWS)	6
2.1. Cloud Computing	6
2.2. Big Data e implementación en AWS	7
2.2.1. Paradigma de las 5 V con AWS	7
2.2.2. Arquitectura de procesamiento distribuido	8
3. Fase de análisis	8
3.1. Analisis del dataset	8
3.2. Descripción tecnica del dataset	8
3.3. Requisitos del sistema	9
3.3.1. Requisito Funcional	10
3.3.2. Requisitos no funcionales	10
4. Fase de diseño	10
4.1. Arquitectura general del sistema	10
4.2. Diseño del Data Lake	11
4.3. Diseño del modelo de datos relacional	11
4.4. Diseño de la logica de procesamiento en Spark	11

5. Fase de implementación	12
5.1. Despliegue de la base de datos	12
5.2. Configuración del S3	21
5.3. Proceso ETL	22
5.3.1. Transformaciones	29
5.4. Procesamiento distribuido en Amazon EMR	34
6. Fase de pruebas	36
6.1. Pruebas de conectividad y seguridad	36
6.2. Prueba de integridad y calidad del dato	37
6.3. Prueba de cómputo distribuido	37
6.4. Validación del modelo	37
7. Analítica avanzada y minería de datos	37
7.1. Consulta 1	40
7.2. Consulta 2	42
7.3. Consulta 3	43
7.4. Consulta 4	44
7.5. Consulta 5	46
7.6. Modelo ML	47
8. Comparativa On-Premise vs Cloud	50
8.1. Análisis de costes(CAPEX vs OPEX)	51
8.2. Escalabilidad y Elasticidad	51
8.3. Mantenimiento y operación	51
8.4. Seguridad	51
8.5. Tiempo de despliegue	51
8.6. Complejidad	52
8.7. Recomendación de cada enfoque	52
9. Conclusiones y líneas futuras	52
9.1. Conclusiones	52
9.2. Líneas futuras	53
10.Bibliografía	53
11.Anexo: Diario de Trabajo	54
12.Anexos técnicos	55

1. Introducción

1.1. Contextualización

En las últimas décadas, las grandes ciudades han experimentado un crecimiento exponencial en la generación de datos administrativos y seguridad ciudadana, lo que ha llevado a la creación de grandes cantidades de datos que requieren ser analizados y procesados de manera eficiente. Utilizando técnicas de Big Data y Cloud Computing, es posible analizar y procesar grandes cantidades de datos que permiten hallar patrones delictivos, identificación de zonas de riesgo así como la predicción de futuros eventos.

```
ID,CaseNumber,Date,Block,IUCR,PrimaryType,Description,Location_Description,Arrest,Domestic,Beat,District,Ward,CommunityArea,FBIcode,X_Coordinate,Y_Coordinate,Year,UpdatedOn,Latitude,Longitude,Location
18801165,3A100077,01/01/2017 01:00:00 AM,8000X W DIVISION ST,0460,BATTERY,SIMPLE,BAR OR TAVERN,True,False,1024,18.0,42.0,8.0,888,,,2017,01/08/2017 03:48:20 PM,,,
```

Figura 1: Columnas y primera fila de nuestro dataset

Debido a que el proyecto solicita un dataset que cuente con al menos tres tablas relacionadas, es necesario realizar un proceso de normalización del dataset original. Tras la normalización, nos quedarían estas 3 tablas:

- **Tabla de hechos(Crimes):** Tabla que contiene la información principal del delito, cuenta con las siguientes columnas: 'id', 'case_number', 'date.timestamp', 'block', 'iucr', 'location_id', 'arrest', 'domestic'
- **Tabla de dimensiones(Location):** Tabla que contiene la información de la ubicación del delito, cuenta con las siguientes columnas: 'location_id', 'Location Description', 'District', 'Ward', 'CommunityArea'
- **Tabla de dimensiones(Tipo Crimen):** Tabla que contiene la información del tipo de delito, cuenta con las siguientes columnas: 'iucr', 'primary_type', 'description'

1.2. Justificación

El procesamiento de un dataset como el que vamos a utilizar **Chicago_Crimes_2017** que contiene información detallada sobre tipos de delitos, arrestos y ubicaciones geospaciales, presenta desafíos en términos de procesamiento y análisis de datos que pueden ser más complicados en un entorno On-Premise. Debido a la necesidad de escalabilidad, alta disponibilidad y potencia de cómputo, es justificable el uso de la nube con plataformas como **Amazon Web Services(AWS)**.

1.3. Objetivos

El objetivo principal de este proyecto es diseñar e implementar una arquitectura de datos End-to-End en AWS para el análisis de la criminalidad en Chicago. Para alcanzar este fin se plantean los siguientes objetivos:

- Configurar una base de datos relacional en Amazon RDS para el almacenamiento de los datos.
- Implementar un Data Lake en S3 organizado por zonas (**Raw, Processed, Curated**).
- Automatizar los procesos de extracción y catalogación mediante AWS Glue.
- Realizar procesamiento distribuido utilizando RDD y SparkSQL sobre un cluster de Amazon EMR.
- Desarrollar un modelo de minería de datos para la predicción de incidentes.

1.4. Alcance del proyecto

El proyecto trata el diseño, despliegue y validación de una plataforma analítica integral sobre la infraestructura de **Amazon Web Service (AWS)** para el análisis y predicción de la criminalidad en Chicago. En este proyecto encontramos los siguientes puntos:

- **Gestión de datos relacionales:** Creación de una base de datos utilizando Amazon RDS que cuenta con 3 tablas.
- **ETL:** Realización de un proceso de **Extract, Transform y Load** para migrar los datos a un Data Lake.
- **Análisis:** Ejecución de consultas mediante SparkSQL para analizar el dataset.
- **Predicciones:** Desarrollo de un modelo de minería de datos con SparkMLlib para la clasificación de incidentes delictivos.
- **Visualización de resultados:** Generación de un Jupyter Notebook con gráficas para analizar el dataset.

1.5. Planificación

La ejecución de este proyecto se ha realizado a lo largo de una semana.

1.5.1. Primera fase: Análisis del diseño - 1 día

Durante esta fase se realizaron los siguientes pasos:

- **Estudio del dataset:** Se realizó un estudio detallado del dataset para entender su estructura y contenido.
- **Normalización de datos:** Se realizó la normalización de datos para eliminar la redundancia de datos y asegurar la integridad referencial.

1.5.2. Segunda fase: Implementación de la estructura base - 1 día

Durante esta fase se realizaron los siguientes pasos:

- **Configuración de la base de datos:** Se realizó la configuración de la base de datos utilizando Amazon RDS y la carga de los datos.
- **Configuración del Amazon S3:** Creación del Data Lake y su estructura de carpetas (Raw, Processed, Curated).

1.5.3. Tercera fase: Implementación de la ETL - 1 día

Durante esta fase se realizaron los siguientes pasos:

- **Desarrollo del Job de Glue:** Implementación de la lógica para extraer los datos del RDS al S3, cambio su formato a parquet.
- **Catalogación de datos:** Se realizó la catalogación de los datos para su uso en el Data Lake.

1.5.4. Cuarta fase: Procesamiento distribuido con Amazon EMR - 1 día

Durante esta fase se realizaron los siguientes pasos:

- **Configuración del cluster:** Configuración y lanzamiento del cluster con Amazon EMR.
- **Desarrollo en Spark:** Implementación de transformaciones usando RDD y consultas analíticas con SparkSQL.
- **Analítica avanzada:** Entrenamiento de un modelo de clusterización con SparkMLlib.

1.5.5. Quinta fase: Visualización de resultados - 1 día

Durante esta fase se realizaron los siguientes pasos:

- **Generación de gráficas:** Generación de gráficas para analizar el dataset.

1.5.6. Sexta fase: Documentación - 3 días

Durante esta fase se realizaron los siguientes pasos:

- **Documentación:** Redacción de la memoria técnica del proyecto.

2. Marco teórico (Cloud Computing y Big Data en AWS)

2.1. Cloud Computing

El **cloud computing** es la disponibilidad de recursos de computación bajo demanda (como almacenamiento e infraestructura) como servicios a través de internet. Elimina la necesidad de que las personas y empresas gestionen sus propios recursos físicos y solo paguen por lo que usen.

Hay tres tipos principales de modelos de servicios de cloud computing que puedes seleccionar según el nivel de control, la flexibilidad y la gestión que necesite la empresa:

- **Infraestructura como servicio (IaaS):** La infraestructura como servicio ofrece acceso bajo demanda a servicios de infraestructura de TI, como computación, almacenamiento, redes y virtualización. Ofrece el máximo nivel de control sobre todos los recursos de TI y se asemeja más a los recursos de TI on-premise.
- **Plataforma como servicio(PaaS):** Plataforma como servicio ofrece todos los recursos de hardware y software necesarios para desarrollar aplicaciones en la nube. Con PaaS, las empresas pueden centrarse por completo en el desarrollo de aplicaciones sin la carga que se supe gestionar y mantener la infraestructura subyacente.
- **Software como servicio(SaaS):** El software como servicio proporciona una pila de aplicaciones completa como servicio, desde la infraestructura subyacente hasta el mantenimiento y las actualizaciones del propio software de la aplicación. Una solución SaaS a menudo es una aplicación de usuario final en la que el servicio y la infraestructura los gestiona y mantiene el proveedor de servicios en la nube.

Existen tres tipos distintos de modelos de despliegue de cloud computing:

- **Nube pública:** Las nubes públicas las ejecutan proveedores de servicios. Ofrecen recursos de computación, de almacenamiento y de red por Internet, lo que permite a las empresas acceder a recursos bajo demanda compartidos en función de sus requisitos únicos y de sus objetivos empresariales.
- **Nube privada:** Las nubes privadas se crean, gestionan y pertenecen a una sola organización, y se alojan de forma privada en sus propios centros de datos, lo que comúnmente se denomina "on-premise". Proporcionan más control, seguridad y gestión de datos, al mismo tiempo que permiten que los usuarios internos se beneficien de un conjunto compartido de recursos de computación, almacenamiento y redes.

- **Nube híbrida:** Las nubes híbridas combinan modelos de nube pública y privada con los que las empresas pueden aprovechar los servicios de nube pública y mantener las funciones de cumplimiento y seguridad habituales en arquitecturas de nube privada.

El cloud computing tiene las siguientes ventajas:

- **Flexibilidad:** Gracias a la arquitectura, las empresas y sus usuarios pueden acceder a los servicios en la nube desde cualquier parte del mundo con conexión a internet.
- **Eficacia:** Las empresas pueden desarrollar nuevas aplicaciones e incorporarlas a la fase de producción, sin preocuparse por la arquitectura subyacente.
- **Seguridad:** La seguridad es más sólida debido a la profundidad y amplitud de los mecanismos de seguridad que aplican los proveedores de servicios en la nube.
- **Rentabilidad:** Las empresas solo pagan por los recursos de computación que utilizan.

2.2. Big Data e implementación en AWS

El concepto de Big Data no se refiere únicamente al volumen de la información sino a la incapacidad de las herramientas tradicionales para procesarla, debido a esta razón utilizaremos la plataforma de Big Data junto con alguno de los servicios que nos ofrecen.

2.2.1. Paradigma de las 5 V con AWS

La infraestructura de AWS permite abordar las dimensiones clásicas del Big Data de la siguiente manera:

- **Volumen:** La capacidad de almacenamiento es muy elevada gracias a **Amazon S3** que actúa como núcleo del Data Lake para almacenar todos los registros.
- **Velocidad:** El procesamiento se agiliza mediante el uso de **Amazon EMR**, que permite levantar clústeres de computación en minutos para realizar análisis sobre grandes volúmenes de datos.
- **Variedad:** AWS permite integrar tanto datos estructurados como datos semiestructurados y catalogarlo todo mediante **AWS Glue**.
- **Veracidad:** Los procesos de ETL en Glue se aseguran de la limpieza y normalización del dato antes del análisis.
- **Valor:** Se extrae conocimiento mediante **SparkMLlib** y consultas con **SparkSQL**, transformado registros en información que podemos utilizar para realizar análisis o predicciones.

2.2.2. Arquitectura de procesamiento distribuido

El núcleo de procesamiento de este proyecto reside en Apache Spark ejecutándose en un cluster de Amazon EMR. A diferencia de otros sistemas tradicionales, la integración mediante AWS ofrece:

- **Abstracción de datos:** Spark utiliza RDD (Resilient Distributed Datasets) para manejar la tolerancia a fallos y la computación en memoria, lo que aumenta su velocidad respecto a MapReduce.
- **Gestión de metadatos:** La integración con Glue Data Catalog permite que SparkSQL reconozca las tablas y esquemas definidos en el Data Lake facilitando las consultas complejas.

3. Fase de análisis

3.1. Analisis del dataset

Para este proyecto utilizaremos el dataset **Chicago-Crimes-2017**, este dataset es un subset del sistema CLEAR de la policia de Chicago. Este dataset contiene incidente criminales con la excepción de asesinatos de Enero del año 2017. Este dataset cuenta con un total de 11358 registros distintos, por lo cual es necesario un sistema capaz de procesar todos los datos para su analisis posterior. Los datos se encuentran en formato CSV y cuenta con una gran cantidad de valores relacionados al espacio temporal y geográfico.

3.2. Descripción tecnica del dataset

El dataset cuenta con estas variables:

- **ID:** Identificador único numérico asignado a cada registro de incidente delictivo. Es la Clave Primaria (PK) en nuestra tabla de hechos en Amazon RDS.
- **CaseNumber:** Código alfanumérico único proporcionado por el Departamento de Policía de Chicago para el seguimiento administrativo del caso.
- **Date:** Fecha y hora en la que ocurrió el incidente. Este campo es crítico para las consultas de tendencias temporales y evoluciones mensuales solicitadas[cite: 94, 99].
- **Block:** Dirección anonimizada del bloque donde se produjo el suceso, manteniendo la privacidad de los ciudadanos involucrados.
- **IUCR:** (Illinois Uniform Crime Reporting) Código de cuatro dígitos que clasifica el tipo de delito según el estándar del estado de Illinois.

- **PrimaryType:** Categoría principal del delito (ej. BATTERY, THEFT, DECEPTIVE PRACTICE). Se utilizará para las comparativas entre categorías en SparkSQL[cite: 95].
- **Description:** Subcategoría detallada del tipo de delito primario, ofreciendo mayor contexto sobre la naturaleza del incidente.
- **Location Description:** Descripción del entorno donde ocurrió el delito (ej. STREET, RESIDENCE, APARTMENT).
- **Arrest:** Valor booleano que indica si el incidente resultó en un arresto (True/False). Es una métrica clave para evaluar la eficacia policial.
- **Domestic:** Indica si el incidente fue clasificado como violencia doméstica según la normativa vigente (True/False).
- **Beat:** Identificador de la unidad geográfica mínima de patrullaje policial.
- **District:** Distrito policial encargado de la zona donde se registró el crimen.
- **Ward:** Distrito electoral municipal donde ocurrió el evento.
- **CommunityArea:** Identificador de una de las 77 áreas comunitarias de Chicago, utilizado para análisis estadísticos socioeconómicos.
- **FBICode:** Código de clasificación de delitos según el estándar del FBI para la elaboración de informes uniformes de criminalidad.
- **X Coordinate / Y Coordinate:** Coordenadas del sistema de proyección de Illinois para la ubicación del incidente, esenciales para análisis geoespaciales.
- **Year:** Año del incidente (2017). Campo utilizado para el particionamiento de datos en el Data Lake de Amazon S3.
- **UpdatedOn:** Marca de tiempo de la última actualización del registro en el sistema operacional.
- **Latitude / Longitude:** Coordenadas geográficas estándar (WGS84). Se utilizarán para las visualizaciones cartográficas en el cuaderno Jupyter[cite: 31, 108].
- **Location:** Tupla que agrupa la latitud y longitud para facilitar el consumo de datos por herramientas de visualización externas.

3.3. Requisitos del sistema

El proyecto cumple con los siguientes requisitos:

3.3.1. Requisito Funcional

- **Ingesta de datos:** El sistema extrae los datos desde un sistema relacional (RDS) y los vuelca en un Data Lake.
- **ETL:** Realización de un ETL y conversión a un formato optimizado para Big Data como parquet.
- **Procesamiento analítico:** Ejecución de consultas SQL sobre los datos proporcionados.
- **Minería de Datos:** Implementación de un modelo de Clustering para extraer patrones.

3.3.2. Requisitos no funcionales

- **Escalabilidad:** La arquitectura del proyecto permite el crecimiento del volumen de datos debido a las características que tiene AWS.
- **Seguridad:** Control de accesos mediante roles que ofrece AWS para proteger el Data Lake.
- **Persistencia:** Los resultados analíticos y el modelo entrenado se almacena de forma persistente en zonas específicas del Data Lake.

4. Fase de diseño

4.1. Arquitectura general del sistema

La arquitectura diseñada para este proyecto se basa en un ecosistema de servicios gestionados de Amazon Web Services. El flujo de datos sigue un modelo lineal desde el entorno transaccional hasta la visualización analítica.

- **Capa de origen:** En esta capa usaremos Amazon RDS, para que actúe como el sistema operacional que contiene todos los registros de nuestro dataset.
- **Capa de ingesta y ETL:** Utilización de AWS Glue para la extracción del dataset desde el RDS mediante el uso de crawlers y la utilización de Jobs para la limpieza y carga de los datos en nuestro Data Lake.
- **Capa de almacenamiento:** Utilizaremos un Amazon S3 como nuestro Data Lake, este bucket seguirá la siguiente estructuración (RAW, PROCESSED, CURATED), además tendrá una zona donde guardaremos nuestro modelo entrenado.
- **Capa de cómputo distribuido:** Utilizaremos Amazon EMR para crear el cluster que usará Apache Spark para el procesamiento masivo y minería de datos.

- **Capa de presentación:** Utilizaremos un cuaderno Jupyter mediante Amazon EMR para generar visualizaciones y resultados analíticos.

4.2. Diseño del Data Lake

Para garantizar la integridad y trazabilidad de los datos de nuestro dataset, se utilizará una estructura de carpetas dentro de nuestro Data Lake que permita el flujo de trabajo organizado:

- **Raw:** Almacena los archivos en su estado original cuando son extraídos del RDS, se guardan en formato CSV.
- **Processed:** Contiene los datos cargados tras la limpieza y normalización realizada en AWS Glue. Se utiliza en formato parquet para optimizar el almacenamiento y rendimiento de las consultas.
- **Curated:** Almacena los resultados de las consultas de SparkSQL y los rankings analíticos listos para el consumo final.
- **Models:** En esta zona se guardan los modelos de minería de datos entrenados con SparkMLlib.

4.3. Diseño del modelo de datos relacional

Con el fin de simular un entorno real, el dataset se ha normalizado en tres tablas relacionadas dentro del Amazon RDS:

- **Tabla de hechos:** Esta tabla contiene las métricas principales y las claves foráneas hacia las otras dimensiones.
- **Dimensión de tipos de delitos:** Esta tabla almacena la descripción de los delitos que hay en nuestro dataset mediante el código IUCR y el tipo de delito.
- **Dimensión de localización:** Centraliza los datos de ubicación como District, Ward para evitar redundancias.

4.4. Diseño de la lógica de procesamiento en Spark

El diseño del procesamiento en el cluster EMR se divide en tres niveles de abstracción:

- **Transformaciones:** Se utilizarán funciones map, filter y reduceByKey para la manipulación de bajo nivel de los registros del dataset.
- **Consultas analíticas:** Se utilizarán 5 consultas distintas con SparkSQL para identificar tendencias, hacer rankings y evoluciones temporales.
- **Minería de datos:** Se diseñó un flujo de Machine learning con vectorización de características, división del dataset y entrenamiento de un modelo de clusterización con MLlib.

5. Fase de implementación

En esta fase se detalla la puesta en marcha de la arquitectura diseñada, siguiendo el flujo de datos desde la ingesta hasta el procesamiento distribuido.

5.1. Despliegue de la base de datos

Primero crearemos nuestra base de datos relacional en Amazon RDS utilizando MySQL para actuar como origen transaccional

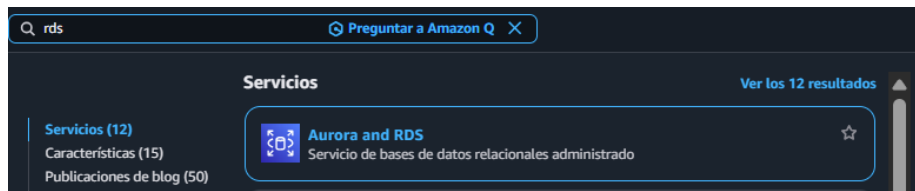


Figura 2: Servicio RDS

Configuramos nuestra base de datos RDS

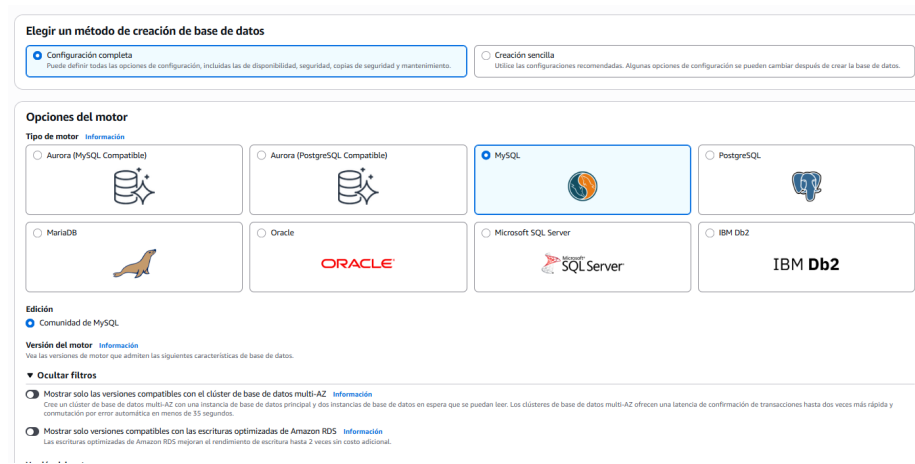


Figura 3: Motor Base de datos

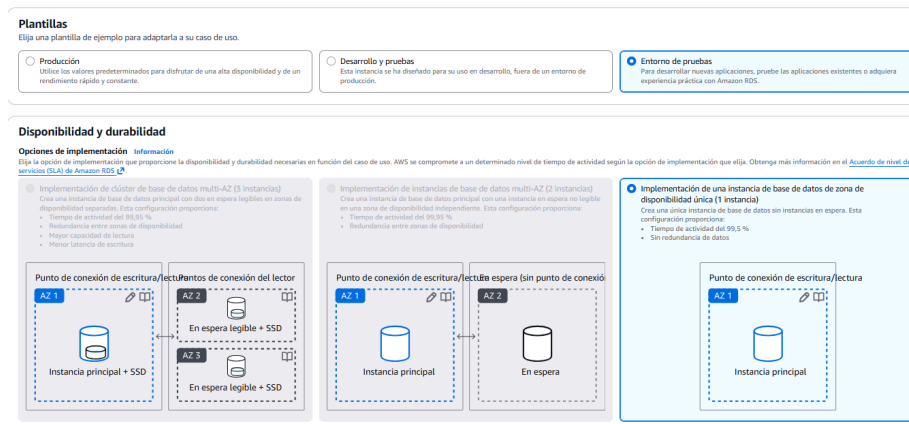


Figura 4: E
lección plantilla

Identificador de instancias de bases de datos Información
Escriba un nombre para la instancia de base de datos. El nombre debe ser único en relación con todas las instancias de base de datos pertenecientes a su cuenta de AWS en la región de AWS actual.

Proyecto-UDS

El identificador de la instancia de base de datos no distingue entre mayúsculas y minúsculas, pero se almacena con todas las letras en minúsculas (como en "mydbinstance"). Restricciones: de 1 a 63 caracteres alfanuméricos o guiones. El primer carácter debe ser una letra. No puede contener dos guiones consecutivos. No puede terminar con un guión.

Configuración de credenciales

Nombre de usuario maestro Información
Escriba un ID de inicio de sesión para el usuario maestro de la instancia de base de datos.

admin

1 a 16 caracteres alfanuméricos. El primer carácter debe ser una letra.

Administración de credenciales
Puede usar AWS Secrets Manager o administrar sus credenciales de usuario maestro.

☐ Administrado en AWS Secrets Manager - más seguro
RDS genera una contraseña y la administra durante todo su ciclo de vida mediante AWS Secrets Manager.

☒ Autoadministrado
Cree su propia contraseña o pída a RDS que cree una contraseña para que pueda administrarla.

☐ Generar contraseña automáticamente
Amazon RDS puede generar una contraseña en su nombre, o bien puede especificar su propia contraseña.

Contraseña maestra Información

Password strength: **Muy strong**

Restricciones mínimas: al menos 8 caracteres ASCII imprimibles. No puede contener ninguno de los siguientes símbolos: / * @

Confirmar la contraseña maestra Información

▼ Configuración de credenciales adicionales

Opciones de autenticación de bases de datos Información

☒ Autenticación con contraseña
Se autentica con las contraseñas de las bases de datos.

☐ Autenticación de bases de datos con contraseña e IAM
Se autentica con las credenciales de usuario y la contraseña de las bases de datos a través de usuarios y roles de AWS IAM.

☐ Autenticación Kerberos y con contraseña
Elija un directorio en el que desea permitir que los usuarios autorizados se autenticquen en esta instancia de base de datos a través de la autenticación Kerberos.

Figura 5: Configuración de credenciales

Con esto, ya tendríamos preparado nuestro RDS para utilizarlo.

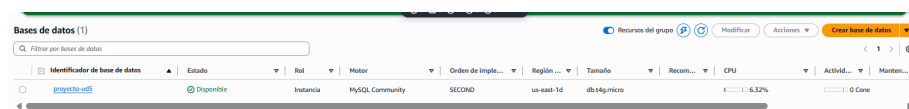


Figura 6: RDS disponible

Lo proximo que haremos será crear una instancia virtual utilizando el servicio

de EC2 que tiene AWS para poder conectarnos a nuestra base de datos en RDS.

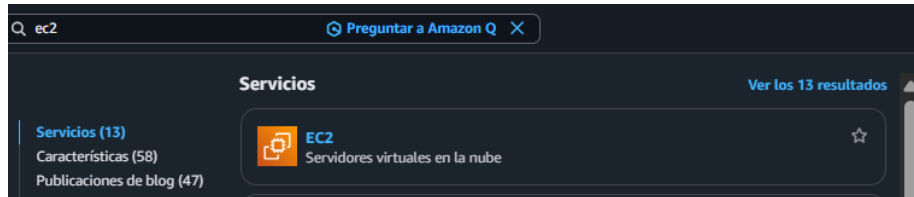


Figura 7: Servicio EC2

A la hora de crear nuestra instancia en EC, le añadiremos el formato .pem para la clave privada, este archivo nos permitirá conectarnos a la instancia de manera segura desde la cloudshell.

A screenshot of the 'Crear par de claves' (Create key pair) dialog in the AWS console. The dialog has a title bar with a close button (X). The main content area is divided into sections. The first section is 'Nombre del par de claves' (Key pair name), with a text input field containing 'ProyectoUD5' and a note: 'Con los pares de claves es posible conectarse a la instancia de forma segura.' (With key pairs, it is possible to connect to the instance securely). Below this is a note: 'El nombre puede incluir hasta 255 caracteres ASCII. No puede incluir espacios al principio ni al final.' (The name can include up to 255 ASCII characters. It cannot include spaces at the beginning or end). The second section is 'Tipo de par de claves' (Key pair type), with two radio button options: 'RSA' (selected) and 'ED25519'. The RSA option has a description: 'Par de claves pública y privada cifradas mediante RSA' (Public and private key pair encrypted using RSA). The ED25519 option has a description: 'Par de claves privadas y públicas cifradas ED25519' (Private and public key pair encrypted using ED25519). The third section is 'Formato de archivo de clave privada' (Private key file format), with two radio button options: '.pem' (selected) and '.ppk'. The .pem option has a description: 'Para usar con OpenSSH' (For use with OpenSSH). The .ppk option has a description: 'Para usar con PuTTY' (For use with PuTTY). At the bottom of the dialog, there is a yellow warning box with a triangle icon and the text: 'Cuando se le solicite, almacene la clave privada en un lugar seguro y accesible del equipo. Lo necesitará más adelante para conectarse a la instancia. Más información' (When prompted, store the private key in a secure and accessible location on your device. You will need it later to connect to the instance. More information). Below the warning box are two buttons: 'Cancelar' (Cancel) and 'Crear par de claves' (Create key pair).

Figura 8: Configuración par clave

Para poder acceder a la instancia de EC2 que hemos creado utilizaremos el

servicio de cloudshell que también ofrece AWS.

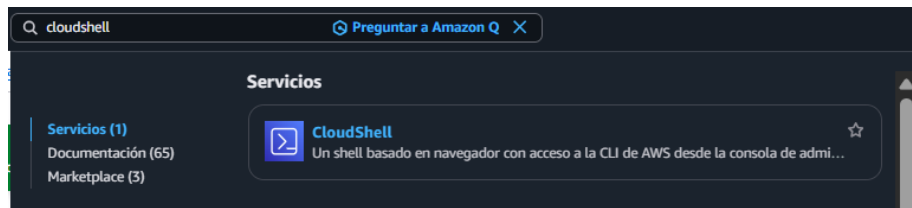


Figura 9: Servicio CloudShell

Subiremos al CloudShell el archivo .pem que hemos descargado al crear nuestra instancia y le otorgaremos permisos.



Figura 10: Subida archivo privado

```
1  chmod 400 "ProyectoUD5.pem"
```

Con el archivo ya subido ya podremos conectarnos mediante ssh, podremos encontrar el comando exacto para acceder a nuestra instancia dentro de la configuración de la propia instancia.

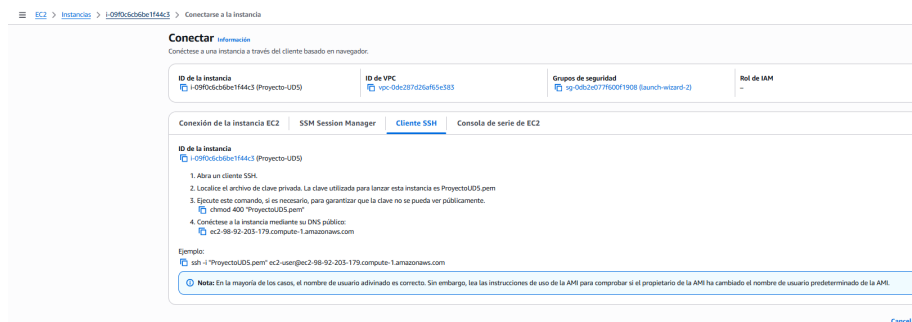


Figura 11: Comando SSH

Comprobamos que podemos conectarnos a la instancia.

Conectar la base de datos de RDS Información

Al conectar una base de datos de RDS a la instancia de EC2, se crean y agregan automáticamente grupos de seguridad para permitir el tráfico entre la base de datos y la instancia de EC2.

ID de la instancia de EC2
i-0c270a82c03ddfe7a

ID de VPC de instancia
vpc-0de287d26af65e383

Seleccione la base de datos de RDS para establecer una conexión con la instancia de EC2.

Rol de base de datos

☐ Clúster
Aplicar grupos de seguridad a todas las instancias de base de datos dentro de un clúster (solo se admiten clústeres regionales)

☒ Instancia
Aplicar grupos de seguridad a instancias de base de datos individuales que no forman parte de un clúster

Base de datos de RDS
Se agrega un grupo de seguridad a la instancia de EC2. También se agrega un grupo de seguridad a la base de datos con una regla de entrada que permite a la instancia de EC2 acceder al puerto de la base de datos.

proyectoud5
Rol: instancia Motor: MySQL Community Region and AZ: us-east-1a

vpc-0de287d26af65e383

[Crear una base de datos de RDS](#)

ⓘ Se le cobrarán tarifas de transferencia de datos porque su instancia EC2 y su base de datos de RDS se encuentran en zonas de disponibilidad diferentes. Para evitar incurrir en estos cargos, deben estar en la misma zona de disponibilidad.

Cancelar
Conectar

Figura 14: Conexión RDS y EC2

Con la conexión realizada ya podremos conectarnos a nuestra base de datos mediante el EC2 y crear nuestras tablas.

```

[ec2-user@ip-172-31-67-254 ~]$ mysql --local-infile=1 -h proyectoud5.cli8ikw6wosh.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 51
Server version: 8.4.7 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
  
```

Figura 15: Conexión con el RDS

Lo proximo que haremos será crear nuestra base de datos y las tres tablas.

```

MySQL [(none)]> CREATE DATABASE chicago_crimes;
Query OK, 1 row affected (0.060 sec)

MySQL [(none)]> USE chichago_crimes;
ERROR 1049 (42000): Unknown database 'chichago_crimes'
MySQL [(none)]> USE chicago_crimes;

```

Figura 16: Creación base de datos

Dentro del dataset que acabamos de crear, crearemos las tres tablas para nuestro dataset.

```

MySQL [chicago_crimes]> CREATE TABLE tipos_crimen(
-> iucr VARCHAR(10) PRIMARY KEY,
-> primary_type VARCHAR(100),
-> description VARCHAR(255)
-> );
Query OK, 0 rows affected (0.095 sec)

```

Figura 17: Creación tabla de dimension de tipo de crimen

```

MySQL [chicago_crimes]> CREATE TABLE ubicaciones (
-> location_id INT AUTO_INCREMENT PRIMARY KEY,
-> location_description VARCHAR(255),
-> district INT,
-> ward INT,
-> community_area INT
-> );
Query OK, 0 rows affected (0.128 sec)

```

Figura 18: Creación tabla de dimensión de ubicaciones

```

MySQL [chicago_crimes]> CREATE TABLE crimenes (
  -> id INT PRIMARY KEY,
  -> case_number VARCHAR(20),
  -> date_timestamp DATETIME,
  -> block VARCHAR(255),
  -> iucr VARCHAR(10),
  -> location_id INT,
  -> arrest BOOLEAN,
  -> domestic BOOLEAN,
  -> FOREIGN KEY(iucr) REFERENCES tipos_crimen(iucr),
  -> FOREIGN KEY(location_id) REFERENCES ubicaciones(location_id)
  -> );
Query OK, 0 rows affected (0.114 sec)

```

Figura 19: Creación tabla de hechos de crímenes

Comprobamos que se hayan creado las tres tablas bien.

```

MySQL [chicago_crimes]> SHOW TABLES;
+-----+
| Tables_in_chicago_crimes |
+-----+
| crimenes                    |
| tipos_crimen                |
| ubicaciones                 |
+-----+
3 rows in set (0.026 sec)

MySQL [chicago_crimes]> DESCRIBE crimenes;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id             | int           | NO   | PRI | NULL    |       |
| case_number    | varchar(20)   | YES  |     | NULL    |       |
| date_timestamp | datetime      | YES  |     | NULL    |       |
| block          | varchar(255)  | YES  |     | NULL    |       |
| iucr           | varchar(10)   | YES  | MUL | NULL    |       |
| location_id    | int           | YES  | MUL | NULL    |       |
| arrest         | tinyint(1)    | YES  |     | NULL    |       |
| domestic       | tinyint(1)    | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.053 sec)

```

Figura 20: Comprobación de las tablas

Cargaremos nuestros datos que habíamos separado en tres tablas con anterioridad para poder usarlo en nuestra instancia de EC2.

Para poder pasar los contenidos en formato CSV de nuestras tablas de la instancia a nuestra base de datos vamos a tener que realizar un cambio a la

configuración del RDS.

Crearemos un grupo de parámetros nuevo para nuestro RDS

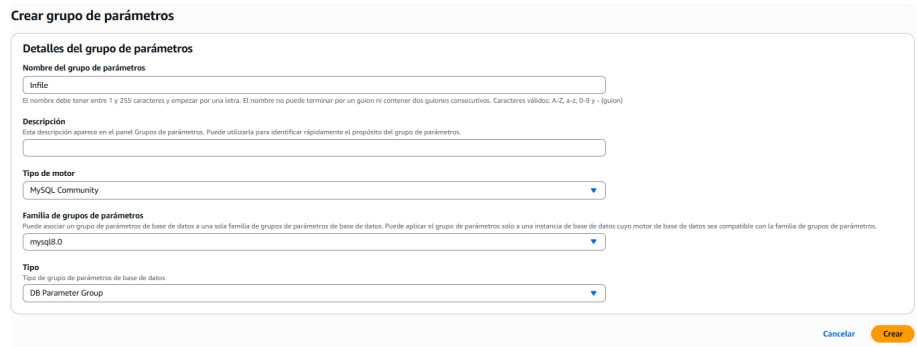


Figura 21: Creación grupo de parametros

Dentro del nuevo grupo de parámetros modificaremos el parametro de local_infile, este nos permitirá cargar datos de un archivo desde nuestro equipo.



☐	Nombre	Valor	Tipo de aplicación	Tipo de datos	Origen
☐	local_infile	1	Dynamic	Boolean	System default

Figura 22: Cambiar configuración local infile

El siguiente paso que haremos será poblar las tablas que acabamos de crear con los datos de nuestro dataset tras la normalización

```
ERROR 1048 (30000): NO database selected
MySQL [(none)]> use chicago_crimes;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [chicago_crimes]> LOAD DATA LOCAL INFILE '/home/ec2-user/tipos_crimen.csv'
-> INTO TABLE tipos_crimen
-> FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
Query OK, 137 rows affected, 1 warning (0.028 sec)
Records: 137 Deleted: 0 Skipped: 0 Warnings: 1
```

Figura 23: Poblar dimensión tipo crimen

```
MySQL [chicago_crimes]> LOAD DATA LOCAL INFILE '/home/ec2-user/ubicaciones.csv'
-> INTO TABLE ubicaciones
-> FIELDS TERMINATED BY ','
-> ENCLOSED BY '"'
-> LINES TERMINATED BY '\n'
-> IGNORE 1 ROWS
-> (location_id, location_description, district, ward, community_area);
```

Figura 24: Poblar dimensión ubicaciones

```
MySQL [chicago_crimes]> LOAD DATA LOCAL INFILE '/home/ec2-user/crimenes.csv'
-> INTO TABLE crimenes
-> FIELDS TERMINATED BY ','
-> ENCLOSED BY '"'
-> LINES TERMINATED BY '\n'
-> IGNORE 1 ROWS
-> (id, case_number, date_timestamp, block, iucr, location_id, arrest, domestic);
```

Figura 25: Poblar tabla de hechos

Comprobamos que todas las tablas se han rellenado con los datos de nuestro dataset.

```
MySQL [chicago_crimes]> SELECT 'Tipos' as Tabla, COUNT(*) as Total FROM tipos_crimen
-> UNION
-> SELECT 'Ubicaciones', COUNT(*) FROM ubicaciones
-> UNION
-> SELECT 'Crimenes', COUNT(*) FROM crimenes;
```

Tabla	Total
Tipos	137
Ubicaciones	3134
Crimenes	11125

3 rows in set (0.077 sec)

Figura 26: Comprobación tablas

Con esto ya tendríamos preparada nuestra base de datos para poder utilizarla.

5.2. Configuración del S3

Para crear nuestro Data Lake, crearemos y configuraremos un bucket de S3 con una estructura jerárquica de carpetas para separar los estados del datos en tres tipos(Raw, Processed, Curated).



Figura 27: Servicio RDS

Configuración general

Región de AWS
EU-IL (Este (Norte de Virginia)) us-east-1

Tipo de bucket [Información](#)

☒ **Uso general**
Recomendado para la mayoría de los casos de uso y patrones de acceso. Los buckets de uso general son del tipo de bucket de S3 original. Permiten una combinación de clases de almacenamiento que almacenan objetos de forma redundante en múltiples zonas de disponibilidad.

☐ **Directorio**
Recomendado para casos de uso de baja latencia. Estos buckets utilizan únicamente la clase de almacenamiento S3 Express One Zone, que proporciona un procesamiento más rápido de los datos dentro de una única zona de disponibilidad.

Nombre del bucket [Información](#)

proyecto-ud5

Los nombres de los buckets deben tener entre 3 y 63 caracteres y ser únicos dentro del espacio de nombres global. Los nombres de los buckets también deben empezar y terminar con una letra o un número. Los caracteres válidos son a-z, 0-9, puntos (.) y guiones (-). [Más información](#)

Copiar la configuración del bucket existente: opcional
Solo se copia la configuración del bucket en los siguientes ajustes.
[Elegir el bucket](#)

Formato: s3://bucket/prefijo

Figura 28: Creación del bucket en S3

Comprobamos que se crea correctamente nuestra estructura de directorios dentro del bucket.

proyecto-ud5 [Información](#)

Objetos | Metadatos | Propiedades | Permisos | Métricas | Administración | Puntos de acceso

Objetos (3) [Copiar URI de S3](#) [Copiar URL](#) [Descargar](#) [Abrir](#) [Eliminar](#) [Acciones](#) [Crear carpeta](#) [Cargar](#)

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el [Inventario de Amazon S3](#) para obtener una lista de todos los objetos de su bucket. Para que otras personas obtengan acceso a sus objetos, tendrá que concederles permisos de forma explícita. [Más información](#)

☐	Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
☐	curated/	Carpeta	-	-	-
☐	processed/	Carpeta	-	-	-
☐	raw/	Carpeta	-	-	-

Figura 29: Comprobación estructura S3

5.3. Proceso ETL

Con nuestro bucket S3 preparado y nuestra base de datos cargada con nuestros datos ya podremos continuar con el ETL, para ello tendremos que utilizar el servicio de AWS Glue.

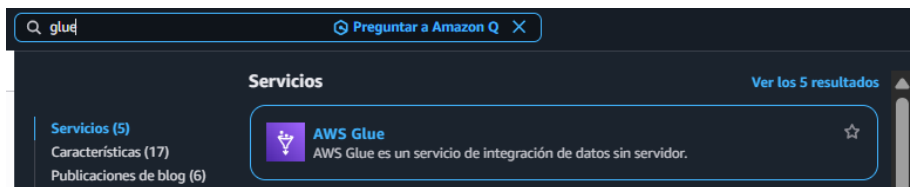


Figura 30: Servicio Glue

Lo primero que haremos dentro de Glue sera crear la conexión con nuestra base de datos para que más tarde nuestro crawler pueda acceder a ella.

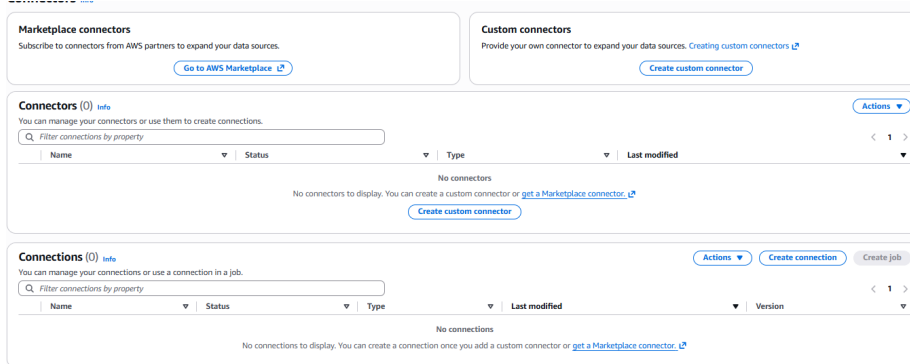


Figura 31: Creación conexión

Para la conexión utilizaremos JDBC.

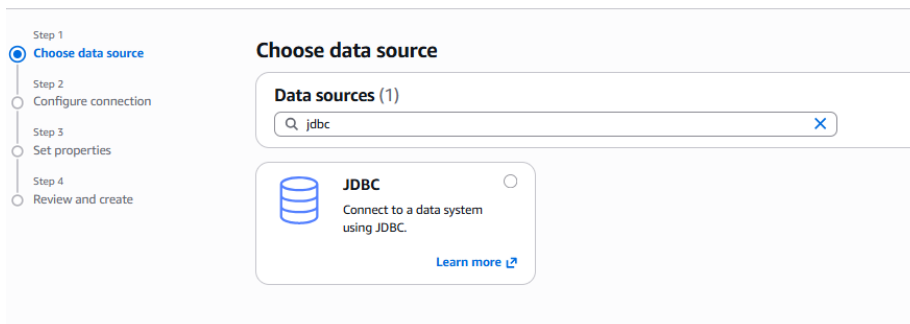


Figura 32: Elección fuente de datos

Configuramos los otros parametros de nuestra conexión como la URL a la que se tiene que conectar, credenciales para poder conectarse, y la VPC.

Configure connection

Connection details

JDBC URL
Use the JDBC protocol to access Amazon Redshift, Amazon RDS, and publicly accessible databases.
jdbc:mysql://projectcloud5.c18kfwosh.us-east-1.rds.amazonaws.com:3306/chicago_crimes
JDBC syntax for most database engines is jdbc:protocol://host:port/databaseName.

JDBC Driver Class name - optional
Type a custom JDBC driver class name for the crawler to connect to the data source.

JDBC Driver S3 Path - optional
Browse for or enter an existing S3 path to a jar file.
s3://bucket/prefix/object View Browse S3

Credential type
☒ Username and password
☐ AWS Secrets Manager

Username
admin

Password

Network options
If your AWS Glue job needs to run on Amazon Elastic Compute Cloud (EC2) instances in a virtual private cloud (VPC) subnet, you must provide additional VPC-specific configuration information.

VPC Info
Choose the virtual private cloud that contains your data source.
vpc-0de287d26af65e383

Subnet Info
Choose the subnet within your VPC.
subnet-046f8a888dec3252
amazonec2-us-east-1:537423485963:subnet/subnet-046f8a888dec3252
zone: us-east-1b

Security groups Info
Choose one or more security groups to allow access to the data store in your VPC subnet. Security groups are associated to the ENI attached to your subnet. You must choose at least one security group with a self-referencing inbound rule for all TCP ports.
Choose one or more security group
sg-0565b0ac6a9df5b29 default

Cancel Previous Next

Figura 33: Configuración conexión

Con la conexión ya creada, podemos seguir creando el crawler. Este crawler se encargará de automatizar la creación del esquema de la base de datos para que otros servicios puedan entender el contenido de las tablas.

Crawlers (0) Info
View and manage all available crawlers.

March 11, 2024 at 09:43:12 Action Run Create crawler

Filter crawlers

Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from last run
No resources No resources to display.						

Figura 34: Crear crawler

Le añadimos la fuente de datos de donde tiene que acceder

Add data source

Data source

Choose the source of data to be crawled.

JDBC

Connection

Select a connection to access the data sources below.

Conexión Proyecto

Clear selection

Add new connection

Include path

chicago_crimes

You can substitute the percent (%) character for a schema or table. For databases that support schemas, enter MyDatabase/MySchema/% to match all tables in MySchema within MyDatabase. Oracle Database and MySQL don't support schema in the path; instead, enter MyDatabase/%. For Oracle database without SSL, MyDatabase can be either the system identifier (SID) or the service name (SERVICE_NAME). For Oracle database with SSL, MyDatabase must be the service name (SERVICE_NAME).

Additional metadata - optional

Select additional metadata properties for the crawler to crawl.

☐ Exclude files matching pattern

Cancel

Add a JDBC data source

Figura 35: Añadir fuente de datos

Crearemos una base de datos en AWS Glue Data Catalog

Create a database

Create a database in the AWS Glue Data Catalog.

Database details

Name

db_chicago_crimes

Database name is required, in lowercase characters, and no longer than 255 characters.

Description - optional

Enter text

Descriptions can be up to 2048 characters long.

Database settings

Location - optional

Set the URI location for use by clients of the Data Catalog.

An S3 location is required for managed tables and Zero-ETL integrations.

Cancel

Create database

Figura 36: Creación base de datos

Configuramos el crawler añadiendole la base de datos a la que tiene que apuntar.

Figura 37: Configuración target BD crawler

Para que el crawler funcione hay que modificarle la regla de entrada.

Figura 38: Modificar regla entrada

Comprobamos si nuestro crawler puede acceder a la base de datos utilizando el test connection.

Figura 39: Comprobación conexión

Lanzamos nuestro crawler y comprobamos que funciona correctamente

Figura 40: Ejecución del crawler

Comprobamos el resultado del crawler en nuestra Data Catalog de Glue.

db_chicago_crimes Last updated (UTC) March 11, 2026 at 10:33:43 [Edit](#) [Delete](#)

Database properties

Name db_chicago_crimes	Description -	Location -	Created on (UTC) March 11, 2026 at 09:56:15
----------------------------------	-------------------------	----------------------	---

Tables (5) Last updated (UTC) March 11, 2026 at 10:33:43 [Delete](#) [Add tables using crawler](#) [Add table](#)

View and manage all available tables.

Filter tables

☐	Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statistics
☐	chicago_crimes_crimes	db_chicago_crimes	chicago_crimes.crimes	mysql	-	-	View data quality	View statistics
☐	chicago_crimes_tipos_crimen	db_chicago_crimes	chicago_crimes.tipos_crimen	mysql	-	-	View data quality	View statistics
☐	chicago_crimes_ubicaciones	db_chicago_crimes	chicago_crimes.ubicaciones	mysql	-	-	View data quality	View statistics

Figura 41: Comprobación del crawler

Con nuestros datos ya preparados dentro del Data Catalog, podemos empezar con el ETL lo primero que tenemos que hacer es crear un Job en Glue.

Your jobs (0) [info](#) Last updated (UTC) March 11, 2026 at 10:33:43 [Actions](#) [Run job](#)

Filter jobs by property

☐	Job name	Type	Created by	Last modified	AWS Glue version	Action
No jobs						
You have not created a job yet.						
Create job from a blank graph						

Figura 42: Creación job ETL

Nuestro ETL tendrá la siguiente estructura.

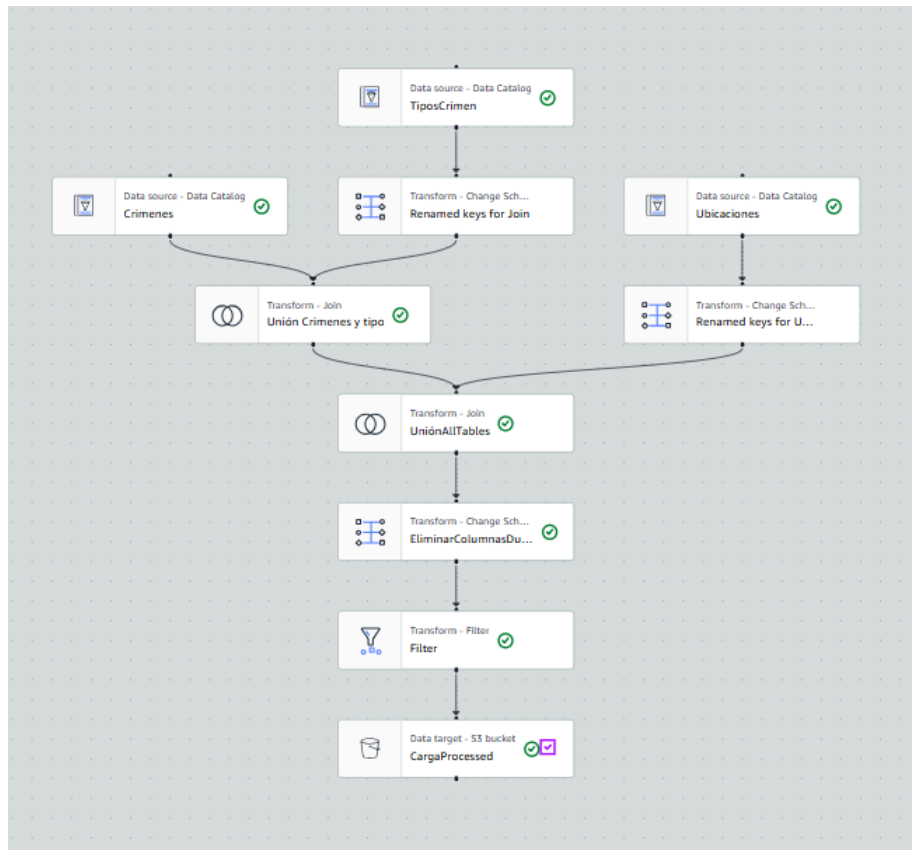


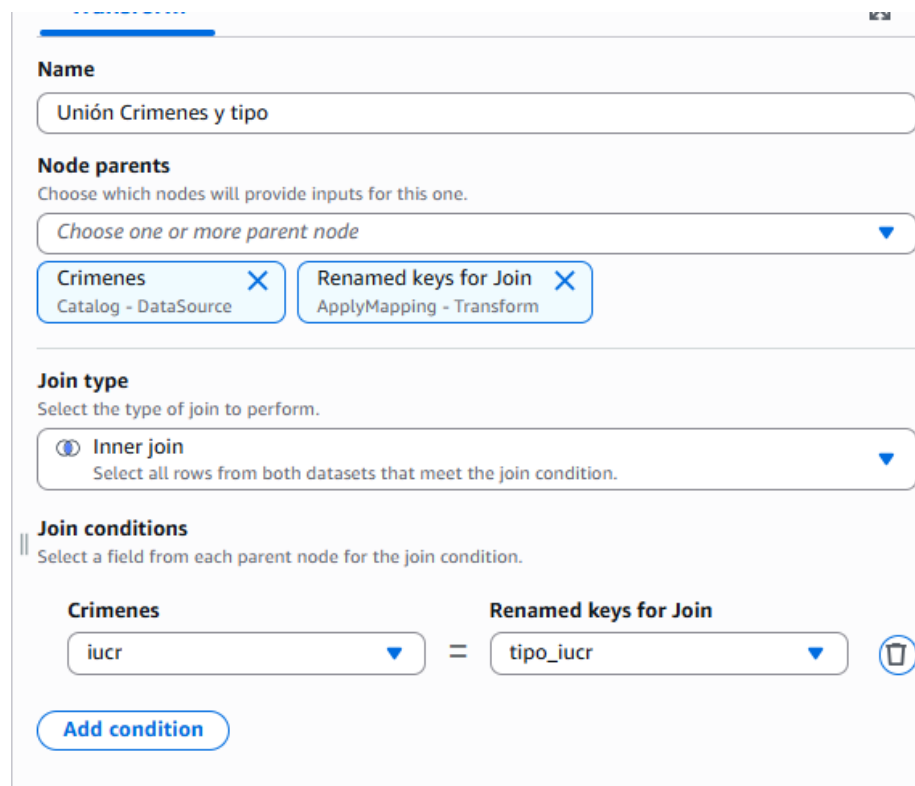
Figura 43: ETL

La estructura que tiene nuestro ETL es la siguiente:

- Primero, tenemos 3 fuentes de datos que apuntan cada una a una tabla en nuestro Data Catalog
- Seguido de ello, existen dos join para unir las 3 tablas en una sola.
- Añadimos una transformación de Change Squema para borrar las claves que se repitan.
- También añadimos una transformación de tipo filter que se encarga de eliminar los valores que incumplan ciertas condiciones, para evitar que haya nulos en las columnas importantes.
- Por ultimo tenemos un destino, que apuntará a la carpeta processed dentro de nuestro S3

5.3.1. Transformaciones

Realizaremos primeros las dos uniones



The screenshot shows a configuration window for a data transformation node. The window has a title bar with a blue bar and a close button. The main content area is divided into several sections:

- Name:** A text input field containing "Unión Crímenes y tipo".
- Node parents:** A section with the instruction "Choose which nodes will provide inputs for this one." and a dropdown menu showing "Choose one or more parent node". Below this are two buttons: "Crímenes" (with a close button) and "Renamed keys for Join" (with a close button). The "Crímenes" button has a tooltip "Catalog - DataSource" and the "Renamed keys for Join" button has a tooltip "ApplyMapping - Transform".
- Join type:** A section with the instruction "Select the type of join to perform." and a dropdown menu showing "Inner join" with a description "Select all rows from both datasets that meet the join condition." and a close button.
- Join conditions:** A section with the instruction "Select a field from each parent node for the join condition." and two dropdown menus. The first dropdown is labeled "Crímenes" and shows "iucr". The second dropdown is labeled "Renamed keys for Join" and shows "tipo_iucr". There is an equals sign between the two dropdowns and a trash icon to the right. Below this is a button labeled "Add condition".

Figura 44: Unión crímenes y tipo crímenes

Transform

Name

UniónAllTables

Node parents
Choose which nodes will provide inputs for this one.

Choose one or more parent node

Unión Crímenes y tipo

Join - Transform

Renamed keys for UniónAllTables

ApplyMapping - Transform

Join type
Select the type of join to perform.

Inner join

Select all rows from both datasets that meet the join condition.

Join conditions
Select a field from each parent node for the join condition.

Unión Crímenes y tipo

location_id

=

Renamed keys for UniónAllTables

right_location_id

Add condition

Figura 45: Unión todas las tablas

Eliminamos las claves repetidas

30

Filter condition | info

Specify your filter condition by choosing the key, operator, and entering a value.

Key	Operation
iucr	matches
☐ Show full path	
Value	
.+	
Use a regex string.	
Remove	

Key	Operation
location_id	>
☐ Show full path	
Value	
1	
Remove	

Key	Operation
id	>
☐ Show full path	
Value	
1	
Remove	

Figura 47: Filtro nulos

Por último realizamos la carga de los datos dentro de la carpeta processed de nuestro S3.

Data target properties - CargaProcessed

Name
CargaProcessed

Node parents
Choose which nodes will provide inputs for this one.
Choose one or more parent node

Filter X
Filter - Transform

Format
Parquet

After you save your job, it will use Glue Studio's optimized Parquet writer.

Compression Type
Snappy

S3 Target Location
Choose an S3 location in the format s3://bucket/prefix/object/ with a trailing slash (/).
s3://proyecto-ud5/processed/ View Browse S3

Data Catalog update options Info
Choose how you want to update the Data Catalog table's schema and partitions. These options will only apply if the Data Catalog table is an S3 backed source.

- ☒ Do not update the Data Catalog
- ☐ Create a table in the Data Catalog and on subsequent runs, update the schema and add new partitions
- ☐ Create a table in the Data Catalog and on subsequent runs, keep existing schema and add new partitions

Figura 48: Carga datos procesados

Ejecutamos nuestro ETL que realizará los cambios necesarios y convertira los csv en formato parquet.

Job runs (1/1) Info

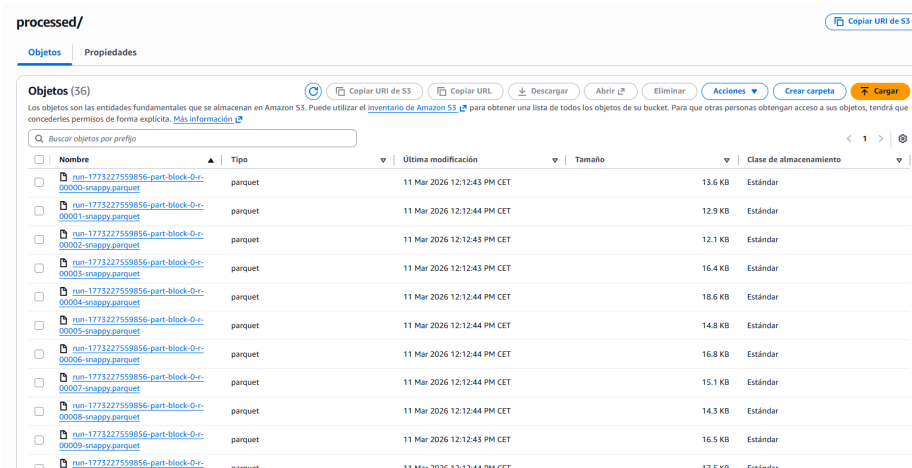
Q. Filter job runs by property

Last updated (UTC) March 11, 2026 at 11:13:10 View details Stop job run Troubleshoot with AI

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPU)	Worker type	Glue version
Succeeded	0	03/11/2026 12:10:03	03/11/2026 12:12:55	2 m 36 s	10 DPUs	G.1X	5.0

Figura 49: Ejecucción ETL

Comprobamos que los resultados del ETL se haya guardado correctamente en la carpeta que le hemos marcado.



Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
nm-1773227559856-part-block-0-00000-snappy.parquet	parquet	11 Mar 2026 12:12:43 PM CET	13.6 KB	Estándar
nm-1773227559856-part-block-0-00001-snappy.parquet	parquet	11 Mar 2026 12:12:44 PM CET	12.9 KB	Estándar
nm-1773227559856-part-block-0-00002-snappy.parquet	parquet	11 Mar 2026 12:12:43 PM CET	12.1 KB	Estándar
nm-1773227559856-part-block-0-00003-snappy.parquet	parquet	11 Mar 2026 12:12:43 PM CET	16.4 KB	Estándar
nm-1773227559856-part-block-0-00004-snappy.parquet	parquet	11 Mar 2026 12:12:44 PM CET	18.6 KB	Estándar
nm-1773227559856-part-block-0-00005-snappy.parquet	parquet	11 Mar 2026 12:12:44 PM CET	14.8 KB	Estándar
nm-1773227559856-part-block-0-00006-snappy.parquet	parquet	11 Mar 2026 12:12:44 PM CET	16.8 KB	Estándar
nm-1773227559856-part-block-0-00007-snappy.parquet	parquet	11 Mar 2026 12:12:44 PM CET	15.1 KB	Estándar
nm-1773227559856-part-block-0-00008-snappy.parquet	parquet	11 Mar 2026 12:12:44 PM CET	14.3 KB	Estándar
nm-1773227559856-part-block-0-00009-snappy.parquet	parquet	11 Mar 2026 12:12:45 PM CET	16.5 KB	Estándar
nm-1773227559856-part-block-0-00010-snappy.parquet	parquet	11 Mar 2026 12:12:44 PM CET	17.6 KB	Estándar

Figura 50: Comprobación resultados ETL

5.4. Procesamiento distribuido en Amazon EMR

Para realizar el procesamiento distribuido utilizaremos el servicio de Amazon EMR.

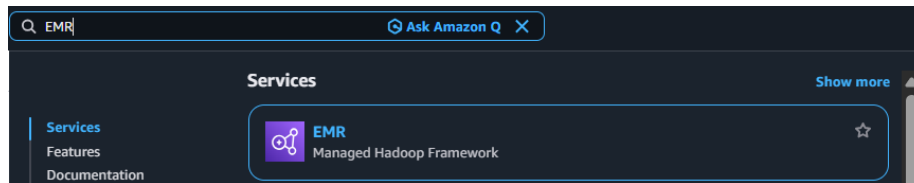


Figura 51: Servicio EMR

Configuraremos nuestro cluster para tener un nodo maestro y 2 nodos esclavos

Create cluster

▼ Name and applications - required

Name your cluster and choose the applications that you want to install on your cluster.

Name

ClusterProyectoUD5

Amazon EMR release

emr-7.12.0

Application bundle

Spark Interactive

Core Hadoop

Flink

HBase

Presto

Trino

Custom

☐ AmazonCloudWatchAgent 1.300052.2
 ☐ Flink 1.20.0
 ☐ HBase 2.6.2

☐ HCatalog 3.1.3
 ☒ Hadoop 3.4.1
 ☒ Hive 3.1.3

☐ Hue 4.11.0
 ☒ JupyterEnterpriseGateway 2.6.0
 ☐ JupyterHub 1.5.0

☒ Livy 0.8.0
 ☐ Oozie 5.2.1
 ☐ Phoenix 5.2.1

☐ Pig 0.17.0
 ☐ Presto 0.287
 ☒ Spark 3.5.6

☐ Tez 0.10.2
 ☐ Trino 476

☐ Zeppelin 0.11.1
 ☐ ZooKeeper 3.9.3

AWS Glue Data Catalogue settings

Use the AWS Glue Data Catalog to provide an external metastore for your application.

☐ Use for Hive table metadata
 ☐ Use for Spark table metadata

Operating system options

☒ Amazon Linux release
 ☐ Customised Amazon Machine Image (AMI)

☒ Automatically apply the latest Amazon Linux updates

Summary

Name and applications - required

Name

ClusterProyectoUD5

Amazon EMR release

emr-7.12.0

Application bundle

Spark Interactive (Hadoop 3.4.1, Hive 3.1.3, JupyterEnterpriseGateway 2.6.0, Livy 0.8.0, Spark 3.5....)

Cluster configuration - required

Instance groups

Primary (r8g.xlarge), Core (r8g.xlarge), Task (r8g.xlarge)

Cluster scaling and provisioning - required

Provisioning configuration

Core size: 1 instance

Configure IAM roles

You must choose a service role and instance profile before you create this cluster.

Choose IAM roles

Cancel

Create cluster

Figura 52: Configuración cluster

También tenemos que configurar los roles necesarios para el cluster.

35

▼ **Identity and Access Management (IAM) roles - *required*** [Info](#)

Choose or create a service role and instance profile for the EC2 instances in your cluster.

Amazon EMR service role [Info](#)

The service role is an IAM role that Amazon EMR assumes to provision resources and perform service-level actions with other AWS services.

☒ **Choose an existing service role**
Select a default service role or a custom role with IAM policies attached so that your cluster can interact with other AWS services.

☐ **Create a service role**
Let Amazon EMR create a new service role so that you can grant and restrict access to resources in other AWS services.

Service role

LabRole
▼

↺

EC2 instance profile for Amazon EMR

The instance profile assigns a role to every EC2 instance in a cluster. The instance profile must specify a role that can access the resources for your steps and bootstrap actions.

☒ **Choose an existing instance profile**
Select a default role or a customised instance profile with IAM policies attached so that your cluster can interact with your resources in Amazon S3.

☐ **Create an instance profile**
Let Amazon EMR create a new instance profile so that you can specify a customised set of resources for it to access in Amazon S3.

Instance profile

EMR_EC2_DefaultRole
▼

↺

Custom automatic scaling role - *optional*

When a custom automatic scaling rule triggers, Amazon EMR assumes this role to add and terminate EC2 instances.

[Find out more](#)

Custom automatic scaling role

Choose an IAM role
▼

↺

Create an IAM role

Figura 53: Configuración de roles

6. Fase de pruebas

Durante esta fase se detallan los procesos de verificación que se han realizado para asegurar la robustez, seguridad e integridad del dato a lo largo de todo el proceso analítico en AWS.

6.1. Pruebas de conectividad y seguridad

Primero comprobamos la conexión del crawler pueda acceder a los datos.

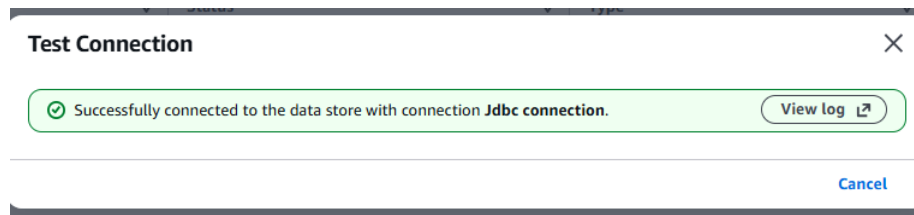


Figura 54: Comprobación conexión

Comprobamos que Amazon EMR tenga los roles necesarios para leer la carpeta Processed y escribir en la carpeta Curated

6.2. Prueba de integridad y calidad del dato

- Se realizó un conteo total de las filas en la tabla origen dentro del RDS y se comparó con el número de registros en el Data Catalog
- Se verificó que los tipos de datos inferidos coinciden con el diseño previo.

6.3. Prueba de cómputo distribuido

- Se validó el correcto funcionamiento de las transformaciones de bajo nivel.
- Se ejecutaron cinco consultas analíticas, para comprobar la velocidad de respuesta.

6.4. Validación del modelo

- Se comprobó la correcta división de los datos en conjuntos de entrenamiento y prueba.
- Se generaron informes de precisión para el modelo implementado.

7. Análítica avanzada y minería de datos

Para poder realizar la analítica avanzada utilizaremos un cuaderno Jupyter. Para ello, crearemos primero un workspace.

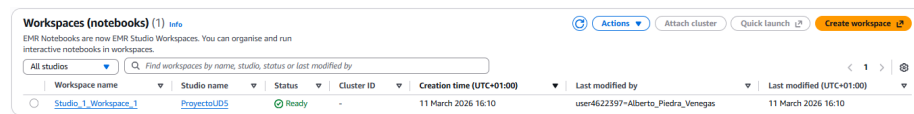


Figura 55: Creación espacio de trabajo

Para poder utilizar el cluster que hemos creado en EMR en nuestro cuaderno de Jupyter, es necesario conectarlo.

Compute

To run your notebooks, attach your workspace to a compute layer. All notebooks in the Workspace share the same compute. Make sure that your EMR Studio user role has appropriate permissions for your selected compute type. [Learn more](#)

▼ **Compute type**

☐ EMR Serverless application

☒ EMR on EC2 cluster

☐ EMR on EKS cluster

EMR on EC2 cluster ⓘ

ProyectoUD5SBD (j-2ZYPCD8... ▼ ↻

Runtime role ⓘ

If you have configured your cluster for runtime roles, you must choose an IAM role for Amazon EMR to assume when it runs your notebooks.

Choose a runtime role ▼ ↻

Attach **Detach**

Figura 56: Conexión notebook - cluster

Con la conexión ya añadida, podemos empezar con la analítica, lo primero que haremos será instalar los paquetes necesarios.

```

import os
import sys

# Esto asegura que Spark use el mismo entorno virtual que el Notebook
os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

Last executed at 2026-03-11 17:35:25 in 37.67s

VBox()
Starting Spark application

ID      YARN Application ID  Kind  State  Spark UI  Driver log  User  Current session?
6  application_1773241180052_0007  pyspark  idle  Link  Link  None  ✓

FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),...
SparkSession available as 'spark'.
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),...

sc.install_pypi_package("numpy")
sc.install_pypi_package("matplotlib")

```

Figura 57: Instalación paquetes

Lo siguiente que haremos será crear una sesión de Spark y añadirle la ruta para que pueda acceder a nuestros datos procesados.

```

1 from pyspark.sql import SparkSession
2
3 spark = SparkSession.builder.appName("ProyectoChicago"
4     ).getOrCreate()
5 path = "s3://proyecto-ud5/processed/"
6 df = spark.read.parquet(path)
7 df.cache()
8 print("Datos cargados correctamente.")

```

Lo siguiente que haremos sera utilizar la interfaz de bajo nivel de Spark para procesar datos de una manera funcional.

```

1 def es_arresto(fila):
2     return fila['arrest'] == True
3
4 def crear_par_crimen(fila):
5     return (fila['primary_type'], 1)
6
7 def sumar_cantidades(a, b):
8     return a + b
9
10 rdd_crimes = df.rdd
11
12 arrests_rdd = rdd_crimes.filter(es_arresto) \
13     .map(crear_par_crimen) \
14     .reduceByKey(sumar_cantidades)
15
16 print("Resultado RDD:")

```

```

17 for registro in arrests_rdd.take(5):
18     print(registro)

```

Este script filtra la base de datos para quedarse con los arrestos, y luego cuenta cada tipo de crimen e imprime los primeros 5 resultados.

```

Last executed at 2026-03-11 17:35:56 in 5.28s
VBox()
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),...
Resultado RDD:
('PUBLIC PEACE VIOLATION', 34)
('BURGLARY', 6)
('WEAPONS VIOLATION', 134)
('ASSAULT', 115)
('HOMICIDE', 1)

```

Figura 58: RDD

7.1. Consulta 1

Realizaremos la primera consulta

```

1 consulta1 = """
2 SELECT HOUR(date_timestamp) as hora, count(*) as
   total_crimes
3 FROM chicago_crimes
4 GROUP BY hora
5 ORDER BY hora
6 """
7 spark.sql(consulta1).show(24)

```

hora	total_crmenes
0	449
1	342
2	314
3	251
4	184
5	203
6	230
7	278
8	374
9	472
10	482
11	516
12	635
13	577
14	577
15	563
16	628
17	622
18	667
19	631
20	606
21	586
22	515
23	403

Figura 59: Resultado primera consulta

Con esta consulta podemos analizar como se distribuye la actividad delictiva a lo largo del día, esto nos puede ofrecer información muy interesante ya que podemos llegar a conclusiones sobre las horas más tranquilas, o al contrario, las horas que mas delitos hay para que la policia puede aumentar las patrullas durante esos momentos del día.

```

1 grafico1 = spark.sql(consulta1).toPandas()
2
3 plt.figure(figsize=(12, 6))
4 plt.bar(pdf['hora'], pdf['total_crmenes'], color='
    skyblue', edgecolor='navy')
5
6 # 3. Personalización
7 plt.title('Crmenes por hora del dia')
8 plt.xlabel('Hora')
9 plt.ylabel('Cantidad de Crmenes')
10 plt.xticks(grafico1['hora'])
11 plt.grid(axis='y', linestyle='--', alpha=0.7)
12

```

```
13 %matplotlib plt
```

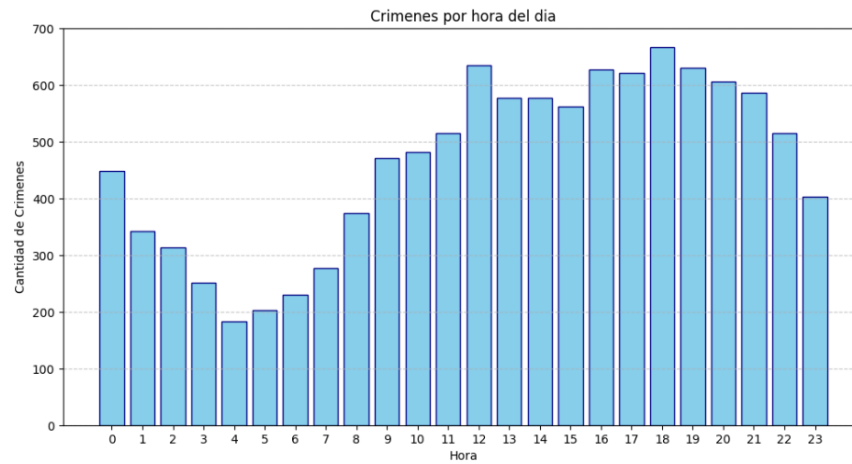


Figura 60: Visualización primera consulta

7.2. Consulta 2

```
1 consulta2 = """SELECT primary_type as tipo_arresto,
2     count(*) as total,
3     round(avg(CAST(arrest AS INT))*100, 2) as
4     porcentaje_arresto
5 FROM chicago_crimes
6 GROUP BY primary_type
7 ORDER BY total DESC
8 LIMIT 10
9 """
10 spark.sql(consulta2).show(10)
```

tipo_arresto	total	porcentaje_arresto
THEFT	2483	9.38
BATTERY	1954	18.42
CRIMINAL DAMAGE	1303	4.99
OTHER OFFENSE	826	21.67
ASSAULT	763	15.07
MOTOR VEHICLE THEFT	709	3.39
BURGLARY	667	0.9
DECEPTIVE PRACTICE	622	3.86
ROBBERY	585	1.2
NARCOTICS	367	100.0

Figura 61: Resultado segunda consulta

Con esta consulta podemos notar 3 conclusiones clave:

- Se pueden ver cuales son los 10 delitos más frecuentes en la ciudad de Chicago
- La tasa de eficacia, los delitos que tienen alto porcentaje, suelen darse en el momento, mientras los que tienen un menor porcentaje es debido a que el autor puede huir y es más difícil de localizar.
- La discrepancia entre el total y el porcentaje de arresto nos dice que aunque el robo sea el que más se comete, no tiene tanto arresto, esto nos dice donde los policias tienen más dificultades y se necesitan más recursos.

```

1 arrest_data = spark.sql("SELECT arrest, count(*) as
    total FROM chicago_crimes GROUP BY arrest").
    toPandas()
2
3 plt.figure(figsize=(6, 6))
4 plt.pie(arrest_data['total'], labels=['No Arrestado',
    'Arrestado'], autopct='%1.1f%%', colors=['
    lightcoral', 'lightgreen'])
5 plt.title('Efectividad Policial: Proporción de
    Arrestos')
6 %matplotlib plt

```

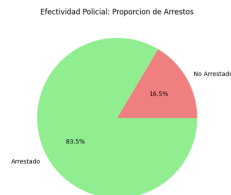


Figura 62: Visualización segunda consulta

7.3. Consulta 3

```

1 consulta3 = """SELECT location_description as lugares,
    count(*) as total
2 FROM chicago_crimes
3 GROUP BY location_description
4 ORDER BY total DESC
5 LIMIT 10"""
6
7 spark.sql(consulta3).show(10)

```

lugares	total
STREET	2643
RESIDENCE	1960
APARTMENT	1577
SIDEWALK	679
OTHER	441
PARKING LOT/GARAG...	392
RESTAURANT	306
SMALL RETAIL STORE	272
ALLEY	223
RESIDENCE-GARAGE	215

Figura 63: Resultado tercera consulta

Esta consulta nos muestra, las ubicaciones donde más crímenes se cometen, gracias a ellas podemos llegar a las siguientes conclusiones:

- Ya que la ubicación predominante es en la calle, podemos llegar a la conclusión de que la criminalidad es predominantemente callejera.
- Ya que es predominantemente callejera, la policía podría reducir el número haciendo más patrullas o colocando cámaras de vigilancia en las calles.

7.4. Consulta 4

```

1 consulta4 = """SELECT district AS distrito, count(*)
2     as total,
3     RANK() OVER (ORDER BY count(*) DESC) as
4     ranking_peligrosidad
5 FROM chicago_crimes
6 GROUP BY distrito
7 ORDER BY ranking_peligrosidad ASC
8 LIMIT 10
9 """
10 spark.sql(consulta4).show(10)

```

distrito	total	ranking_peligrosidad
11	767	1
8	729	2
6	727	3
25	703	4
12	619	5
4	569	6
18	567	7
7	547	8
1	519	9
10	517	10

Figura 64: Resultado cuarta consulta

Gracias a esta consulta podemos saber, que distritos son los que hay más crímenes, y ahí saber a áreas de la ciudad debe la policía destinar más recursos.

Además también se podría comparar el total de delitos de los 10 primeros distritos con el total de delitos y comprobar si están muy concentrados los delitos en unos pocos distritos o si están repartidos de forma equitativa.

```

1 grafico4 = spark.sql(consulta2).toPandas()
2
3 plt.figure(figsize=(10, 6))
4 plt.bar(grafico4['tipo_arresto'], grafico4['total'],
5         color='steelblue')
6 plt.xticks(rotation=45, ha='right')
7 plt.title('Top 10 Tipos de Crímenes en Chicago')
8 plt.ylabel('Cantidad de Incidentes')
9 plt.tight_layout()
%matplotlib plt

```

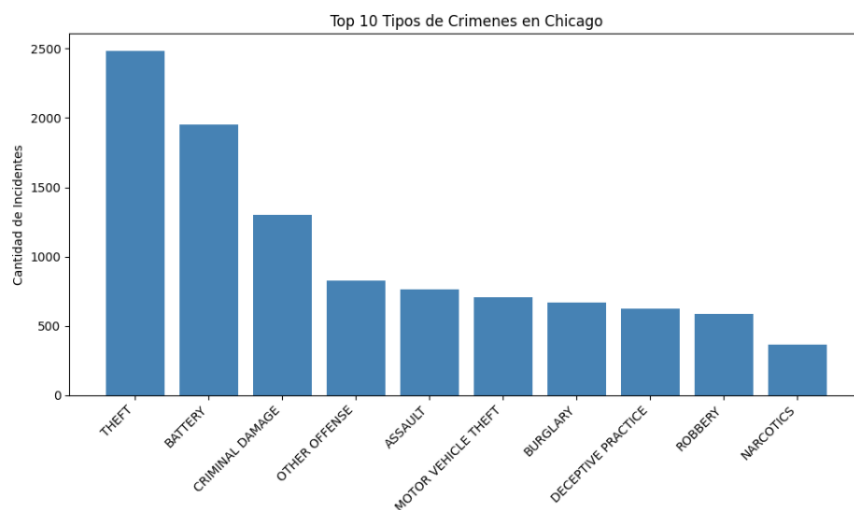


Figura 65: Visualizacion cuarta consulta

7.5. Consulta 5

```

1 consulta5 = """SELECT MONTH(date_timestamp) as mes,
2     count(*) as total
3 FROM chicago_crimes
4 GROUP BY mes
5 ORDER BY mes
6 """
7 spark.sql(consulta5).show(10)

```

```

+---+---+
|mes|total|
+---+---+
|  1|11105|
+---+---+

```

Figura 66: Resultado quinta consulta

Con esta consulta podemos saber en que meses se cometen más delitos, lo que nos permite saber que meses, la policia debe hacer más patrullas.

```
1 grafico5 = spark.sql(consulta5).toPandas()
2
3 plt.figure(figsize=(10, 6))
4 plt.bar(grafico5['mes'], grafico5['total'], color='
  steelblue')
5 plt.xticks(rotation=45, ha='right')
6 plt.title('Top 10 Tipos de Crimenes en Chicago')
7 plt.ylabel('Cantidad de Incidentes')
8 plt.tight_layout()
9 %matplotlib plt
```

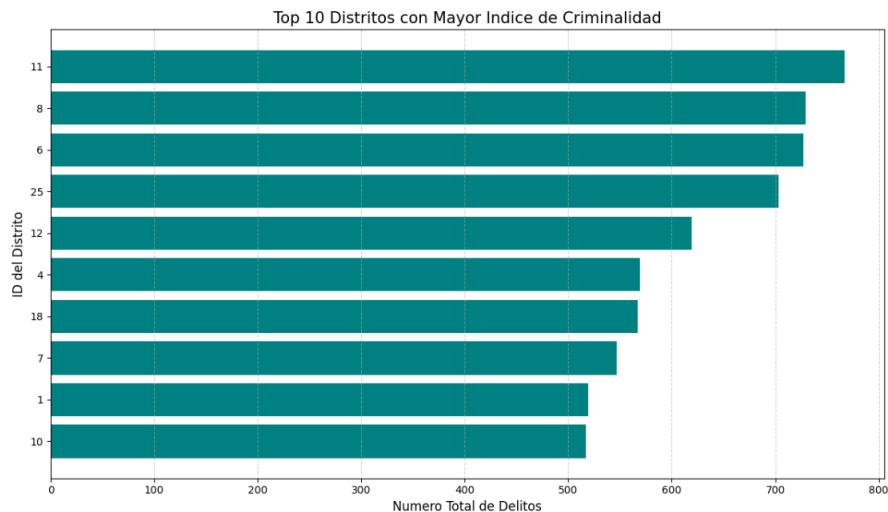


Figura 67: Quinta Visualizacion

7.6. Modelo ML

Para este proyecto realizaremos un modelo de clustering para poder agrupar los distritos de la ciudad de Chicago en base a los delitos que se cometen, y que la policia sepa donde deberia dedicar más recursos.

Primeros lo que haremos será vectorizar los datos y utilizar el Standar Scaler para estandarizar los datos que vamos a utilizar y que los que tengan números más grandes no ganen más importancia que otros datos.

```
1 from pyspark.ml.clustering import KMeans
2 from pyspark.ml.feature import VectorAssembler,
  StandardScaler
3
```

```

4 df_modelo = df.withColumn("hour", F.hour("
    date_timestamp"))
5
6 data_limpiar = df_modelo.select("latitude", "longitude"
    , "hour").na.drop()
7
8 assembler = VectorAssembler(
9     inputCols=["latitude", "longitude", "hour"],
10     outputCol="features_unscaled"
11 )
12 data_vector = assembler.transform(data_limpiar)
13
14 scaler = StandardScaler(inputCol="features_unscaled",
15     outputCol="features", withStd=True, withMean=True)
16 scaler_model = scaler.fit(data_vector)
17 final_data = scaler_model.transform(data_vector)

```

Lo siguiente que haremos será dividir nuestro dataset en datos de entrenamiento (80 %) y datos de test (20 %)

```

1 train_data, test_data = final_data.randomSplit([0.8,
    0.2], seed=123)

```

Utilizaremos este script para calcular cual es mejor valor del hiperparametro K para utilizar en nuestro modelo.

```

1 cost_array = []
2 K_values = range(2, 11)
3
4 for k in K_values:
5     kmeans = KMeans(k=k, seed=100)
6     model = kmeans.fit(final_data)
7     cost_array.append(model.summary.trainingCost)
8
9
10 mejoras = [cost_array[i] - cost_array[i+1] for i in
    range(len(cost_array)-1)]
11
12
13 mejor_k = 2
14 for i in range(1, len(mejoras)):
15     if mejoras[i] < mejoras[i-1] * 0.5:
16         mejor_k = K_values[i]
17         break
18
19 print(f"El valor óptimo sugerido para K es: {mejor_k}"
    )

```

Sabiendo ya el mejor valor para nuestro hiperparametro ya podemos crear nuestro modelo.

```

1 from pyspark.ml.evaluation import ClusteringEvaluator
2
3 kmeans_final = KMeans(k=mejor_k, seed=1)
4 modelo_final = kmeans_final.fit(final_data)
5
6 predictions = modelo_final.transform(final_data)
7 evaluator = ClusteringEvaluator()
8 silhouette = evaluator.evaluate(predictions)
9
10 print(f"Silhouette Score final: {silhouette}")
11 print("Muestra de resultados (Agrupación por ubicación
    y hora):")
12 predictions.select("district", "community_area", "hour",
    "prediction").show(10)

```

```

Silhouette Score final: 0.5346935075668996
Muestra de resultados (Agrupación por ubicación y hora):
+-----+-----+-----+-----+
|district|community_area|hour|prediction|
+-----+-----+-----+-----+
|2|41|21|1|
|2|41|14|1|
|2|41|11|4|
|2|41|16|1|
|2|41|9|4|
|2|41|22|1|
|2|41|23|1|
|2|41|14|1|
|8|57|15|1|
|5|53|9|4|
+-----+-----+-----+-----+
only showing top 10 rows

```

Figura 68: Resultado del modelo

El silhouette score es un valor que nos indica la calidad de nuestro modelo, cuanto mas cercano a 1, mejor es nuestro modelo.

Lo siguiente que haremos será guardar nuestro modelo dentro del bucket de S3.

```

1 path_curated = "s3://proyecto-ud5/curated/"
2
3 predictions.write.mode("overwrite").parquet(
    path_curated)
4
5 print(f"Resultados guardados exitosamente en: {
    path_curated}")
6
7
8 path_models = "s3://proyecto-ud5/Models/KMean/"

```

```

9
10
11 model.save(path_models)
12
13 print(f"Modelo cluster guardado exitosamente en: {
    path_models}")

```

[illegible]

Figura 69: Resultado de las predicciones

The screenshot shows the KMeans model page in the DataCamp interface. The page has a header with the KMeans logo and a 'Copy ID' button. Below the header is a navigation bar with 'Objects' and 'Properties' tabs. The 'Objects' tab is active, showing a list of objects. The objects are listed in a table with columns: Name, Type, Last modified, Size, and Storage class. The objects are 'kmeans' and 'init'. The 'init' object is highlighted in blue. Below the table, there is a section for 'Find objects in profile' with a search bar and a list of objects.

Name	Type	Last modified	Size	Storage class
kmeans	kmeans	10/10/2019	1.0 KB	Standard
init	init	10/10/2019	1.0 KB	Standard

Figura 70: Resultado de la guardado del modelo

Por ultimo generamos una visualizacion para ver como se agrupan los clusters según las variables que hemos seleccionado.

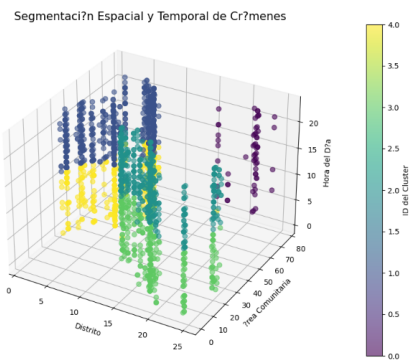


Figura 71: Visualizacion del modelo

8. Comparativa On-Premise vs Cloud

En esta sección se realizara un análisis técnico y estratégico comparando la infraestructura tradicional basada en centros de datos locales(On-Promise) frente a la solución implementada en la nube(Cloud).

8.1. Análisis de costes(CAPEX vs OPEX)

- **Arquitectura On-Premise:** Requiere una inversión de capital inicial elevada para la adquisición de servidores físicos, almacenamiento y networking, con independencia del uso real del clúster.
- **Arquitectura Cloud:** Se rige por un modelo de gastos operativos, donde se paga exclusivamente por el tiempo de computación de los nodos de Amazon EMR y el espacio ocupado.

8.2. Escalabilidad y Elasticidad

- **Arquitectura On-Premise:** La escalabilidad es limitada y rígida. Ampliarla, implica procesos de compra, instalación física y configuración de red que pueden durar un tiempo considerable.
- **Arquitectura Cloud:** Permite una escalabilidad horizontal elástica, es posible aumentar o disminuir el número de nodos de trabajo en un cluster en cuestión de minutos.

8.3. Mantenimiento y operación

- **Arquitectura On-Premise:** El equipo de mantenimiento y operación es mayor, ya que se requiere un equipo de profesionales para el mantenimiento de los servidores, la red y el almacenamiento.
- **Arquitectura Cloud:** El equipo de mantenimiento es menor debido que la administración de la infraestructura se encarga el proveedor.

8.4. Seguridad

- **Arquitectura On-Premise:** La seguridad es mayor, ya que no confías tus datos a un proveedor, además se pueden añadir protocolos de seguridad, para proteger mejor los datos.
- **Arquitectura Cloud:** Los servidores cloud son un arma de doble filo. Tienes acceso a recursos avanzados de seguridad y ciertas certificaciones de cumplimiento. Pero tus datos y la información de los clientes se almacenarán en un servidor externo, lo que aumenta los riesgos de vulnerabilidades.

8.5. Tiempo de despliegue

:

- **Arquitectura On-Premise:** El tiempo necesario para poner en marcha un clúster es mayor, debido a la logística de adquisición de hardware y la configuración de la red.

- **Arquitectura Cloud:** La arquitectura cloud permite desplegar un clúster en menos de una hora.

8.6. Complejidad

- **Arquitectura On-Premise:** Presenta una complejidad mayor debido a que se debe realizar la instalación manual de todo el stack de Big Data, además requiere un conocimiento profundo en la configuración de redes, sistemas de archivos distribuidos.
- **Arquitectura Cloud:** La complejidad de instalar el software desaparece ya que AWS presenta entorno ya preparados, pero aparece la complejidad de la configuración lógica y de seguridad.

8.7. Recomendación de cada enfoque

A pesar del auge de la nube, la infraestructura local sigue siendo la opción ganadora en escenarios muy específicos:

- **Soberanía de datos y cumplimiento estricto:** Es mejor elegir On-premise cuando por ley los datos utilizados no pueden cruzar fronteras físicas o residir en servidores compartidos.
- **Cargas de trabajo constantes:** Si una empresa tiene un cluster de Spark que funciona a una gran capacidad durante mucho tiempo, a largo plazo será más barato que el cloud debido a que el hardware se amortiza.

Cloud es la opción ideal en la gran mayoría de casos debido a:

- **Velocidad:** Para empezar a utilizar un sistema distribuido no es necesario, la instalación y configuración de los servidores físicos.
- **Costes baratos para proyectos experimentales:** Para realizar proyectos experimentales donde se procesan una cantidad de registros y se apaga el cluster costando muy poco, es mejor que el modelo On-premise.

9. Conclusiones y líneas futuras

9.1. Conclusiones

Tras la finalización del proyecto he podido llegar a las siguientes conclusiones:

- El potencial que tiene el procesamiento distribuido para manejar y analizar grandes cantidades de datos, respecto a los sistemas tradicionales.
- La facilidad de implementar y desplegar un clúster en la nube, con la capacidad de escalar horizontalmente.

- La eficiencia del formato columnar como es parquet para la optimización de los tiempos de lectura y escritura de datos.
- La implementación de algoritmos de aprendizaje para poder extraer información valiosa de los datos.

9.2. Líneas futuras

Aunque este proyecto ya está terminado, hay varias líneas futuras que se pueden implementar:

- Ingesta de datos en tiempo real, para poder capturar y procesar los incidentes nada más se registren en el sistema policial.
- Implementación de un dashboard para que los usuarios puedan realizar consultas visuales sin necesidad de conocimientos de SQL o Python
- Añadir datos que puedan ser útiles a nuestro dataset como datos meteorológicos, eventos públicos que haya habido en Chicago, para poder analizar si hay algún tipo de correlación externa que influya en la tasa de delitos.

10. Bibliografía

Referencias

- [1] Cloud computing – <https://cloud.google.com/learn/what-is-cloud-computing?hl=es>
- [2] AWS EMR – <https://docs.aws.amazon.com/emr/>
- [3] AWS S3 – <https://docs.aws.amazon.com/s3/>
- [4] AWS Glue – <https://docs.aws.amazon.com/glue/>
- [5] On-Premise vs Cloud – <https://blog.softexpert.com/es/on-premise-vs-cloud/>
- [6] On-Premise vs Cloud (OVHcloud) – <https://www.ovhcloud.com/es-es/learn/onprem-vs-cloud/>
- [7] Apache Spark SQL – <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- [8] Origen dataset – https://github.com/ricardomariscalquintero/bda/blob/main/Chicago_Crimes_2017.csv

11. Anexo: Diario de Trabajo

Este anexo detalla cronológicamente las actividades realizadas para el desarrollo de la plataforma analítica en AWS, siguiendo los requisitos de integración y procesamiento distribuido establecidos.

Fecha	Descripción de la Tarea
10/03/2026	Elección del dataset Chicago Crimes 2017, diseño de la arquitectura cloud (RDS, S3, Glue, EMR).
10/03/2026	Creación de la base de datos operacional en Amazon RDS y configuración del bucket S3 para el Data Lake.
10/03/2026	Ingesta de datos hacia la zona raw de S3 y creación del proceso ETL mediante AWS Glue.
11/03/2026	Despliegue del clúster Amazon EMR y ejecución del procesamiento distribuido de datos con Spark.
11/03/2026	Análisis avanzado con SparkSQL, implementación del modelo con Spark MLlib y generación de visualizaciones en Jupyter.
14/03/2026	Redacción de la memoria técnica del proyecto utilizando el sistema de composición LaTeX.
15/03/2026	Revisión y corrección de la memoria técnica del proyecto.

Cuadro 1: Registro diario de actividades realizadas durante el desarrollo del proyecto.

12. Anexos técnicos

Nuestro dataset cuenta con estas columnas:

- **ID:** Identificador único numérico asignado a cada registro de incidente delictivo en la base de datos.
- **CaseNumber:** Código alfanumérico único asignado por el Departamento de Policía de Chicago (CPD) para el seguimiento del caso.
- **Date:** Fecha y hora exacta en la que ocurrió el incidente, fundamental para el análisis de series temporales y tendencias[cite: 94, 99].
- **Block:** Dirección anonimizada del bloque donde ocurrió el incidente (ej. "0000X W DIVISION ST") para preservar la privacidad.
- **IUCR:** (Illinois Uniform Crime Reporting) Código de cuatro dígitos que identifica el tipo de delito según el estándar estatal.
- **PrimaryType:** Categoría principal del delito (ej. BATTERY, THEFT, NARCOTICS), esencial para las comparativas por categorías en SparkSQL[cite: 95].
- **Description:** Descripción detallada o subcategoría del tipo de delito primario (ej. "SIMPLE", "DOMESTIC BATTERY").
- **Location Description:** Descripción del tipo de lugar donde se produjo el evento (ej. "BAR OR TAVERN", "STREET", "RESIDENCE").
- **Arrest:** Variable booleana (True/False) que indica si el incidente resultó en un arresto inmediato.
- **Domestic:** Variable booleana que indica si el incidente fue clasificado como violencia doméstica conforme a la Ley de Violencia Doméstica de Illinois.
- **Beat:** Identificador del "Beat", la unidad geográfica más pequeña de patrullaje policial en Chicago.
- **District:** Distrito policial donde ocurrió el incidente.
- **Ward:** El distrito electoral municipal (concejalía) donde se registró el delito.
- **CommunityArea:** Área comunitaria oficial de la ciudad (Chicago tiene 77 áreas definidas) utilizada para análisis estadísticos socioeconómicos.
- **FBICode:** Código de clasificación de delitos según el estándar del FBI para informes uniformes de criminalidad (UCR).
- **X Coordinate / Y Coordinate:** Coordenadas geográficas en el sistema de proyección transversal de Mercator para mapeo preciso.

- **Year:** Año de ocurrencia del delito (en este dataset, 2017), utilizado para filtrados y particionamiento en el Data Lake[cite: 11, 41].
- **UpdatedOn:** Fecha y hora de la última actualización del registro en el sistema operacional.
- **Latitude / Longitude:** Coordenadas geográficas estándar (WGS84) para su uso en herramientas de visualización y SIG[cite: 67, 106].
- **Location:** Tupla que combina latitud y longitud para facilitar la representación geoespacial en mapas.

Para transformar el archivo plano(CSV) en un modelo relacional eficiente, se ha aplicado la Tercera forma normal, para eliminar la redundancia de datos y asegurar la integridad referencial, para ello utilizaremos este script y separaremos cada tabla en un archivo de texto plano(CSV).

```

1
2 import pandas as pd
3
4 file_path = 'C:/Users/alber/OneDrive/Escritorio/IABD/
5             SBD/Proyecto 5/Chicago_Crimes_2017.csv'
6 df = pd.read_csv(file_path)
7
8 df.columns = df.columns.str.strip()
9
10 print("Iniciando normalización de datos...")
11
12 tipos_crimen = df[['IUCR', 'PrimaryType', 'Description
13                   ']].drop_duplicates(subset=['IUCR'])
14 tipos_crimen.columns = ['iucr', 'primary_type', '
15                         description']
16 tipos_crimen.to_csv('tipos_crimen.csv', index=False)
17 print(f"Creado tipos_crimen.csv con {len(tipos_crimen)
18       } registros.")
19
20 ubicaciones = df[['Location Description', 'District',
21                  'Ward', 'CommunityArea']].drop_duplicates()
22 ubicaciones = ubicaciones.reset_index(drop=True)
23 ubicaciones['location_id'] = ubicaciones.index + 1
24 cols = ['location_id', 'Location Description', '
25         District', 'Ward', 'CommunityArea']
26 ubicaciones = ubicaciones[cols]
27 ubicaciones.columns = ['location_id', '
28                       location_description', 'district', 'ward', '
29                       community_area']
30 ubicaciones.to_csv('ubicaciones.csv', index=False)

```

```

23 print(f"Creado ubicaciones.csv con {len(ubicaciones)}
    registros.")
24
25
26 df_final = df.merge(ubicaciones,
27                     left_on=['Location Description', '
    District', 'Ward', 'CommunityArea'],
28                     right_on=['location_description',
    'district', 'ward', 'community_area'],
29                     how='left')
30
31 crimenes = df_final[['ID', 'CaseNumber', 'Date', '
    Block', 'IUCR', 'location_id', 'Arrest', 'Domestic'
    ]]
32 crimenes.columns = ['id', 'case_number', '
    date_timestamp', 'block', 'iucr', 'location_id', '
    arrest', 'domestic']
33
34 crimenes['arrest'] = crimenes['arrest'].astype(int)
35 crimenes['domestic'] = crimenes['domestic'].astype(int
    )
36 crimenes['date_timestamp'] = pd.to_datetime(crimenes['
    date_timestamp']).dt.strftime('%Y-%m-%d %H:%M:%S')
37
38 crimenes.to_csv('crimenes.csv', index=False)
39 print(f"Creado crimenes.csv con {len(crimenes)}
    registros.")

```

Script para la creación del ETL:

```

1 import sys
2 from awsglue.transforms import *
3 from awsglue.utils import getResolvedOptions
4 from pyspark.context import SparkContext
5 from awsglue.context import GlueContext
6 from awsglue.job import Job
7 from awsgluedq.transforms import EvaluateDataQuality
8 import re
9
10 args = getResolvedOptions(sys.argv, ['JOB_NAME'])
11 sc = SparkContext()
12 glueContext = GlueContext(sc)
13 spark = glueContext.spark_session
14 job = Job(glueContext)
15 job.init(args['JOB_NAME'], args)
16
17 # Default ruleset used by all target nodes with data

```

```

    quality enabled
18 DEFAULT_DATA_QUALITY_RULESET = ""
19     Rules = [
20         ColumnCount > 0
21     ]
22 ""
23
24 # Script generated for node TiposCrimen
25 TiposCrimen_node1773225676368 = glueContext.
    create_dynamic_frame.from_catalog(database="
    db_chicago_crimes", table_name="
    chicago_crimes_tipos_crimen", transformation_ctx="
    TiposCrimen_node1773225676368")
26
27 # Script generated for node Ubicaciones
28 Ubicaciones_node1773225679849 = glueContext.
    create_dynamic_frame.from_catalog(database="
    db_chicago_crimes", table_name="
    chicago_crimes_ubicaciones", transformation_ctx="
    Ubicaciones_node1773225679849")
29
30 # Script generated for node Crimenes
31 Crimenes_node1773225622181 = glueContext.
    create_dynamic_frame.from_catalog(database="
    db_chicago_crimes", table_name="
    chicago_crimes_crimenes", transformation_ctx="
    Crimenes_node1773225622181")
32
33 # Script generated for node Renamed keys for Join
34 RenamedkeysforJoin_node1773225901520 = ApplyMapping.
    apply(frame=TiposCrimen_node1773225676368, mappings
    =[("primary_type", "string", "tipo_primary_type", "
    string"), ("description", "string", "
    tipo_description", "string"), ("iucr", "string", "
    tipo_iucr", "string")], transformation_ctx="
    RenamedkeysforJoin_node1773225901520")
35
36 # Script generated for node Renamed keys for Unió
    nAllTables
37 RenamedkeysforUninAllTables_node1773226188235 =
    ApplyMapping.apply(frame=
    Ubicaciones_node1773225679849, mappings=[("
    community_area", "int", "right_community_area", "
    int"), ("district", "int", "right_district", "int")
    , ("ward", "int", "right_ward", "int"), ("
    location_description", "string", "

```

```

    right_location_description", "string"), ("
    location_id", "int", "right_location_id", "int")],
    transformation_ctx="
    RenamedkeysforUninAllTables_node1773226188235")
38
39 # Script generated for node Unión Crimenes y tipo
40 UninCrimenesytipo_node1773225804163 = Join.apply(
    frame1=Crimenes_node1773225622181, frame2=
    RenamedkeysforJoin_node1773225901520, keys1=["iucr"
    ], keys2=["tipo_iucr"], transformation_ctx="
    UninCrimenesytipo_node1773225804163")
41
42 # Script generated for node UniónAllTables
43 UninAllTables_node1773226150027 = Join.apply(frame1=
    UninCrimenesytipo_node1773225804163, frame2=
    RenamedkeysforUninAllTables_node1773226188235,
    keys1=["location_id"], keys2=["right_location_id"],
    transformation_ctx="
    UninAllTables_node1773226150027")
44
45 # Script generated for node EliminarColumnasDuplicadas
46 EliminarColumnasDuplicadas_node1773226539366 =
    ApplyMapping.apply(frame=
    UninAllTables_node1773226150027, mappings=[("
    date_timestamp", "timestamp", "date_timestamp", "
    timestamp"), ("block", "string", "block", "string")
    ], ("tipo_primary_type", "string", "primary_type", "
    string"), ("iucr", "string", "iucr", "string"), ("
    case_number", "string", "case_number", "string"), (
    "right_community_area", "int", "community_area", "
    int"), ("location_id", "int", "location_id", "int")
    ], ("domestic", "boolean", "domestic", "boolean"), (
    "right_location_description", "string", "
    location_description", "string"), ("arrest", "
    boolean", "arrest", "boolean"), ("right_district",
    "int", "district", "int"), ("tipo_description", "
    string", "description", "string"), ("right_ward", "
    int", "ward", "int"), ("id", "int", "id", "int")],
    transformation_ctx="
    EliminarColumnasDuplicadas_node1773226539366")
47
48 # Script generated for node Filter
49 Filter_node1773226234539 = Filter.apply(frame=
    EliminarColumnasDuplicadas_node1773226539366, f=
    lambda row: (bool(re.match(".*", row["iucr"]))) and
    row["location_id"] > 1 and row["id"] > 1),

```

```

    transformation_ctx="Filter_node1773226234539")
50
51 # Script generated for node CargaProcessed
52 EvaluateDataQuality().process_rows(frame=
    Filter_node1773226234539, ruleset=
    DEFAULT_DATA_QUALITY_RULESET, publishing_options={"
    dataQualityEvaluationContext": "
    EvaluateDataQuality_node1773225518935", "
    enableDataQualityResultsPublishing": True},
    additional_options={"dataQualityResultsPublishing.
    strategy": "BEST_EFFORT", "observations.scope": "
    ALL"})
53 CargaProcessed_node1773227094688 = glueContext.
    write_dynamic_frame.from_options(frame=
    Filter_node1773226234539, connection_type="s3",
    format="glueparquet", connection_options={"path": "
    s3://proyecto-ud5/processed/", "partitionKeys":
    []}, format_options={"compression": "snappy"},
    transformation_ctx="
    CargaProcessed_node1773227094688")
54
55 job.commit()

```